

Оглавление

ГЛАВА 1. АНАЛИТИЧЕСКАЯ ЧАСТЬ.....	3
1.1 Характеристика предметной области.....	3
1.2 Анализ сайтов-аналогов	5
1.3 Обоснование создания веб-сайта.....	8
1.4 Постановка задачи	9
1.5 Выбор методов реализации.....	10
ГЛАВА 2. ПРОЕКТНО-ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ.....	13
2.1 Выбор технологий	13
2.1.1 Язык программирования	13
2.1.2 Фреймворк	15
2.1.3 СУБД.....	15
2.1.4 Клиентская часть	16
2.1.5 Инструменты разработки	17
2.2 Проектирование базы данных	17
2.2.1 Описание сущностей	17
2.2.2 Описание связей между сущностями	21
2.2.3 Логическая модель данных.....	22
2.2.4 Физическая модель данных	22
2.3 Архитектура программного обеспечения	24
2.3.1 Общая архитектура системы	24
2.3.3 Диаграмма прецедентов.....	26
2.3.4 Диаграмма классов	28
2.3.5 Диаграмма компонентов.....	30
2.3.6 Диаграмма развёртывания	32
2.4 Алгоритмы работы основных процессов.....	32
2.4.1 Алгоритм регистрации и авторизации пользователя	33
2.4.2 Алгоритм покупки билета	36
2.4.3 Алгоритм модерации мероприятия	38
2.4.4 Алгоритм обработки заявки на возврат билета	40
2.4.5 Алгоритм уведомления подписчиков.....	42
ГЛАВА 3. РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ СИСТЕМЫ	44
3.1 Реализация основных модулей системы.....	44
3.1.1 Модуль аутентификации	44
3.1.2 Модуль продажи и возврата билетов	46
3.1.3 Модуль модерации	47
3.1.4 Модуль уведомлений и планировщик задач	48
3.2 Интерфейс пользователя системы	50
3.2.1 Основные экраны системы	50
3.2.2 Формы ввода данных	50
3.2.3 Панель модерации.....	51

3.3 Тестирование системы	51
3.3.1 Функциональное тестирование.....	51
3.3.2 Тестирование граничных значений	55
3.3.3 Итоги тестирования	55
3.4 Руководство пользователя.....	56
3.4.1 Начало работы	56
3.4.2 Поиск и просмотр мероприятий	56
3.4.3 Покупка билетов	57
3.4.4 Возврат билета	57
3.4.5 Подписка на организации	57
3.5 Руководство администратора и модератора	57
3.5.1 Доступ к панели модерации	57
3.5.2 Модерация организаций	57
3.5.3 Модерация мероприятий	58
3.5.4 Управление пользователями	58
3.5.5 Обработка заявок на возврат	58
3.5.6 Журнал аудита	59
3.5.7 Настройка CRON-задания	59
4. ЭКОНОМИЧЕСКАЯ ЧАСТЬ	59
ТЕХНИЧЕСКОЕ ЗАДАНИЕ.....	63
1. Общие сведения	63
2. Назначение и цели создания системы	63
3. Характеристика объектов автоматизации	64
4. Требования к системе	64
4.1 Требования к системе в целом.....	64
4.2 Требования к функциям (задачам)	66
4.3 Требования к видам обеспечения	67
5. Состав и содержание работ по созданию системы	69
6. Порядок контроля и приёмки системы.....	69
7. Требования к подготовке объекта автоматизации к вводу системы в действие.....	70
8. Требования к документированию.....	70
9. Источники разработки.....	70

ВВЕДЕНИЕ

Область применения разработки — автоматизация процессов публикации культурных мероприятий и реализации билетов для организаций и жителей Магаданской области.

Исходная ситуация характеризуется отсутствием единой цифровой площадки для размещения афиш в регионе. Организаторы мероприятий публикуют информацию в разрозненных группах социальных сетей, принимают заявки по телефону и ведут учёт проданных билетов вручную. Это приводит к риску двойных продаж, потере данных и значительным трудозатратам на рутинные операции. Зрители, в свою очередь, не имеют единого источника актуальной информации о предстоящих событиях.

В рамках данной работы рассматриваются следующие вопросы:

- анализ предметной области и требований к информационной системе;
- выбор технологий и инструментов разработки;
- проектирование структуры базы данных и архитектуры приложения;
- разработка серверной части с реализацией программного интерфейса (API);
- разработка клиентской части с пользовательским интерфейсом;
- реализация модулей аутентификации, продажи билетов, модерации и уведомлений;
- экономическое обоснование разработки.

Предполагаемые результаты разработки — функционирующее веб-приложение «АфишаКолыма», обеспечивающее полный цикл: от регистрации организации и публикации мероприятия до покупки билета пользователем и получения им подтверждения на электронную почту. Система включает личные кабинеты для трёх категорий участников — пользователей, организаций и модераторов, — а также автоматическую рассылку напоминаний о предстоящих мероприятиях.

Актуальность темы обусловлена потребностью жителей и организаторов Магаданской области в современном цифровом инструменте, который сократит время на поиск информации о событиях и устранил ручной труд при продаже и учёте билетов.

ГЛАВА 1. АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1 Характеристика предметной области

Объектом автоматизации является информационно-сервисная платформа для публикации и продажи билетов на культурные мероприятия Магаданской

области. Предметная область охватывает деятельность организаций культуры — театров, концертных залов, выставочных площадок, а также частных организаторов, осуществляющих взаимодействие с конечными потребителями посредством продажи электронных билетов.

В настоящее время процесс информирования населения о культурных событиях региона носит разрозненный характер: организации публикуют анонсы в социальных сетях, на собственных сайтах и в печатных изданиях. Единая точка доступа к актуальной афише отсутствует, что затрудняет поиск мероприятий для пользователей и снижает охват аудитории для организаторов. Продажа билетов осуществляется преимущественно в кассах и через мессенджеры, что не позволяет автоматизировать учёт мест и формировать электронные билеты.

Деятельность в рассматриваемой предметной области включает следующие основные бизнес-процессы.

Управление мероприятиями — создание, редактирование и публикация событий организациями-партнёрами с указанием даты, времени, места проведения, стоимости и количества доступных мест.

Продажа билетов — формирование корзины, подтверждение оплаты и генерация электронного билета с уникальным QR-кодом, обеспечивающим верификацию при входе на мероприятие.

Модерация контента — проверка и одобрение либо отклонение мероприятий уполномоченными сотрудниками платформы перед их публикацией в открытом каталоге.

Обработка возвратов — приём заявок на возврат от пользователей, рассмотрение модератором и изменение статуса билета в соответствии с принятым решением.

Уведомление подписчиков — автоматическая отправка email-уведомлений пользователям, оформившим подписку на конкретную организацию, при публикации нового мероприятия данной организации.

Управление отзывами и рейтингами — возможность пользователей оставлять оценки и текстовые комментарии к посещённым мероприятиям и площадкам, формирование публичного рейтинга на основе оценок.

Система обслуживает три категории участников. Обычные пользователи просматривают афишу, приобретают билеты, оставляют отзывы, управляют списком избранного и оформляют подписки на организации. Организации размещают мероприятия, управляют своим профилем и просматривают статистику продаж в личном кабинете. Модераторы проверяют публикуемый контент, рассматривают заявки на возврат билетов и контролируют общую статистику платформы.

Географический охват системы — Магаданская область Российской Федерации. Региональная специфика предметной области обуславливает необходимость создания собственного решения ввиду отсутствия специализированных платформ для данного рынка среди существующих федеральных аналогов.

1.2 Анализ сайтов-аналогов

В ходе анализа рынка программных решений в области афишных и билетных платформ были рассмотрены три наиболее распространённых продукта: Afisha.ru, Kassir.ru и TimePad. Выбор данных систем для сравнения обусловлен их широкой известностью на российском рынке и функциональным сходством с разрабатываемой платформой.

Afisha.ru — крупнейший российский агрегатор культурных событий с федеральным охватом, функционирующий с 1999 года. Платформа предоставляет пользователям доступ к афише театров, кино, концертов и выставок в крупных городах России, а также обеспечивает онлайн-продажу билетов.

К преимуществам платформы относятся развитая инфраструктура и высокая посещаемость, интеграция с ведущими платёжными системами, наличие

мобильных приложений для iOS и Android, а также развитая система персональных рекомендаций на основе истории посещений.

Вместе с тем платформа обладает рядом существенных недостатков.

Afisha.ru не поддерживает региональные организации малого масштаба и не содержит контента по Магаданской области. Комиссия сервиса составляет до 10–15% от стоимости билета, что является значительной нагрузкой для небольших организаторов. Возможность кастомизации под нужды конкретного региона отсутствует. Модерация заявок новых организаций занимает продолжительное время. Функционал управления возвратами на стороне организатора не предусмотрен, а механизм подписки пользователей на организаторов с уведомлениями отсутствует.

Kassir.ru — федеральная билетная система с разветвлённой сетью офлайн-касс, работающая на российском рынке с 1998 года. Платформа специализируется на продаже билетов на крупные мероприятия: концерты федерального уровня, спортивные соревнования, театральные постановки. Преимуществами системы являются обширная партнёрская сеть, поддержка офлайн-продаж через собственные кассы и интегрированная система печати билетов с защитными элементами.

К недостаткам относятся ориентация исключительно на крупные мероприятия и площадки, необходимость заключения договора и прохождения длительной процедуры верификации при подключении организации, полное отсутствие регионального контента по Магаданской области. Механизм подписки на организаторов и email-уведомлений не реализован. Порог входа для небольших организаций является недопустимо высоким.

TimePad — платформа для организации мероприятий и продажи билетов, ориентированная на небольших организаторов. Пользуется популярностью среди представителей малого бизнеса, некоммерческих организаций и частных лиц.

Преимуществами платформы являются низкий порог входа для организаторов, наличие бесплатного тарифного плана, простой интерфейс создания событий и встроенная аналитика посещаемости.

Вместе с тем TimePad не формирует единого регионального каталога мероприятий — каждое событие существует как отдельная страница без агрегации по географическому признаку. Механизм подписки пользователей на организаторов с автоматическими уведомлениями отсутствует.

Функционал модерации контента ограничен. Площадка не рассматривается как самостоятельная сущность с собственной страницей, историей мероприятий и отзывами.

Результаты сравнительного анализа представлены в таблице 1.1.

Таблица 1.1 — Сравнение существующих аналогов с разрабатываемой системой

Критерий оценки	Afisha.ru	Kassir.ru	TimePad	АфишаКолыма
Региональный фокус	Нет	Нет	Нет	Да
Подписка на организации	Нет	Нет	Нет	Да
Email-уведомления подписчикам	Частично	Нет	Да	Да
Управление возвратами	Ограничено	Ограничено	Нет	Да
Модерация контента	Да	Да	Нет	Да
QR-билеты	Да	Да	Да	Да
Доступность для малых орг.	Нет	Нет	Да	Да

Бесплатное размещение	Нет	Нет	Частично	Да
Площадка как сущность	Нет	Нет	Нет	Да

1.3 Обоснование создания веб-сайта

Проведённый анализ существующих аналогов выявил ряд принципиальных ограничений, не позволяющих применить готовые решения для автоматизации рассматриваемой предметной области.

Первым и ключевым ограничением является полное отсутствие регионального охвата. Ни одна из рассмотренных платформ не представлена в Магаданской области в качестве специализированного ресурса и не содержит актуального контента по местным мероприятиям. Пользователям недоступна единая точка входа для просмотра региональной афиши, а организациям — инструмент для охвата именно местной аудитории.

Вторым ограничением является высокий барьер входа для организаций.

Крупные платформы ориентированы на федеральные структуры и предъявляют требования, неприемлемые для небольших местных организаций культуры: высокие комиссии (до 15% от стоимости билета), длительные процедуры верификации и обязательное заключение официальных договоров. Для организаций Магаданской области данные условия являются экономически нецелесообразными.

Третьим ограничением является отсутствие системы подписок на организаторов. Ни один из рассмотренных аналогов не реализует механизм, позволяющий пользователю подписаться на конкретную организацию и автоматически получать уведомления по электронной почте при публикации нового мероприятия. Данная функция имеет существенное значение для формирования лояльной аудитории у небольших региональных организаторов и повышения повторных продаж.

Четвёртым ограничением является отсутствие полноценного управления возвратами билетов. В существующих решениях данная функция либо полностью отсутствует на стороне организатора, либо реализована в упрощённом виде без процедуры модерации. Отсутствие контролируемого процесса рассмотрения заявок не позволяет обеспечить защиту интересов как пользователей, так и организаторов.

Пятым ограничением является невозможность кастомизации под специфику конкретного региона. Готовые решения не предоставляют возможности адаптации функционала, интерфейса и бизнес-логики под потребности конкретного заказчика.

Таким образом, разработка собственной платформы «АфишаКолыма» является обоснованной и целесообразной. Собственное решение обеспечивает полный контроль над функционалом, персональными данными пользователей и бизнес-процессами, а также позволяет реализовать уникальные функции, отсутствующие во всех рассмотренных аналогах: региональную агрегацию мероприятий, систему подписок с email-уведомлениями и полноценное управление возвратами с процедурой модерации.

1.4 Постановка задачи

На основе проведённого анализа предметной области и выявленных недостатков существующих решений поставлена задача разработки региональной веб-платформы «АфишаКолыма» для публикации культурных мероприятий и продажи электронных билетов в Магаданской области.

Входными данными системы являются регистрационные данные пользователей и организаций (имя, адрес электронной почты, пароль, ИНН), данные мероприятий (название, описание, дата и время проведения, место проведения, стоимость билета, количество доступных мест), изображения мероприятий и площадок, поисковые запросы и параметры фильтрации

каталога, данные подтверждения оплаты билетов, а также текстовые отзывы и числовые оценки пользователей.

Выходными данными системы являются страницы каталога мероприятий и площадок с актуальной информацией, карточки мероприятий с детальным описанием, электронные билеты с уникальным QR-кодом для верификации при входе, email-уведомления о покупке билета, публикации нового мероприятия организации и решении по заявке на возврат, а также отчёты модератора со сводной статистикой платформы.

Разрабатываемая система должна обеспечить взаимодействие трёх категорий пользователей: обычных пользователей, организаций-партнёров и модераторов платформы. Функционал каждой категории разграничивается ролевой моделью на уровне серверной части приложения.

Полный перечень функциональных и технических требований к системе сформулирован в Техническом задании, разработанном в соответствии с ГОСТ 34.602-89 и вынесенном в Приложение А настоящей пояснительной записки.

1.5 Выбор методов реализации

Выбор методов и инструментов реализации системы обусловлен требованиями технического задания, особенностями предметной области и условиями развёртывания на хостинге.

В качестве основного языка серверной части выбран **PHP 8.2** — широко распространённый язык веб-разработки с развитой экосистемой пакетов и поддержкой современных объектно-ориентированных концепций: типизации, именованных аргументов, перечислений (enum). Для ускорения разработки и обеспечения структурированности кода применяется фреймворк **Laravel 11**, реализующий архитектурный паттерн MVC и предоставляющий встроенные механизмы маршрутизации, аутентификации, работы с базой данных и отправки электронной почты. Альтернативные

фреймворки были отклонены: **Symfony** — ввиду большей сложности начальной конфигурации, **CodeIgniter** — ввиду меньшей функциональности из коробки и отсутствия ряда необходимых компонентов.

В качестве системы управления базами данных выбрана **MySQL 8.0** — реляционная СУБД с открытым исходным кодом, широко поддерживаемая хостинг-провайдерами. MySQL обеспечивает поддержку транзакций и внешних ключей (InnoDB), полнотекстового поиска (FULLTEXT-индексы), а также типа ENUM для статусных полей, необходимых для реализации функционала управления мероприятиями и билетами.

Клиентская часть реализована на **нативном JavaScript** без применения тяжёлых фреймворков, таких как React или Vue.js. Данное решение обусловлено отсутствием необходимости в сложном клиентском роутинге, снижением времени загрузки страницы за счёт отказа от дополнительных зависимостей и упрощением процесса развёртывания. Взаимодействие браузера с сервером организовано по принципу REST API: клиент отправляет запросы через Fetch API, сервер возвращает ответы в формате JSON.

Шаблонизация серверной части выполнена средствами **Blade** — встроенного шаблонизатора Laravel. Стилизация реализована на **CSS3** с применением технологий Flexbox и Grid Layout без использования CSS-препроцессоров.

Для аутентификации пользователей применяется **Laravel Sanctum** — официальный пакет для токеной аутентификации, поддерживающий одновременную работу нескольких типов аутентифицируемых сущностей (пользователи и организации как отдельные модели). Отправка транзакционных email-уведомлений осуществляется по протоколу SMTPS через SMTP-сервер хостинг-провайдера с применением HTML-шаблонов Blade. Для генерации QR-кодов на стороне клиента применяется библиотека **qrcode.js**, что исключает необходимость серверной генерации изображений и снижает вычислительную нагрузку.

Разработка ведётся в среде **Visual Studio Code** с использованием системы контроля версий **Git** и платформы **GitHub** для хранения удалённого

репозитория. Локальное окружение организовано посредством **MAMP** (Apache, MySQL, PHP). Развёртывание системы выполняется на хостинге reg.ru под управлением панели ISPmanager.

Сводная информация об используемых инструментах представлена в таблице 1.2.

Таблица 1.2 — Стек технологий разрабатываемой системы

Компонент	Инструмент	Версия
Язык серверной части	PHP	8.2
Фреймворк	Laravel	11
СУБД	MySQL	8.0
Язык клиентской части	JavaScript	ES2021
Шаблонизатор	Blade	встроен в Laravel
Аутентификация	Laravel Sanctum	4.x
Почтовый протокол	SMTPS	порт 465
Система контроля версий	Git / GitHub	2.x
Локальный сервер	MAMP	6.x
Среда разработки	Visual Studio Code	актуальная
Хостинг	reg.ru (ISPmanager)	PHP 8.3, Apache

ГЛАВА 2. ПРОЕКТНО-ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ

2.1 Выбор технологий

2.1.1 Язык программирования

В качестве основного языка серверной части выбран PHP 8.2. PHP является наиболее распространённым языком для разработки веб-приложений: по данным W3Techs, он используется на более чем 77% веб-сайтов с известным серверным языком. Версия 8.2 включает поддержку строгой типизации, именованных аргументов, перечислений (enum), файберов и улучшенной обработки исключений, что позволяет писать надёжный и читаемый код. Альтернативой рассматривался язык Python с фреймворком Django. Однако Python уступает PHP в распространённости на хостинг-платформах:

большинство бюджетных хостингов предоставляют PHP-среду по умолчанию, тогда как настройка Python-окружения требует VPS и дополнительной конфигурации. Это противоречит требованию доступности системы на стандартном виртуальном хостинге.

Node.js с фреймворком Express также рассматривался как альтернатива, однако был отклонён ввиду необходимости ручной реализации большого числа компонентов (аутентификация, ORM, почта), предоставляемых Laravel из коробки, что увеличило бы сроки разработки.

2.1.2 Фреймворк

В качестве фреймворка серверной части выбран Laravel 11 — наиболее популярный PHP-фреймворк, реализующий архитектурный паттерн MVC. Laravel предоставляет следующие встроенные компоненты, использованные в данном проекте:

1. Eloquent ORM — объектно-реляционный маппер для работы с базой данных через объектную модель без написания SQL-запросов вручную;
2. система миграций — механизм версионирования схемы базы данных, позволяющий воспроизводить структуру БД на любом окружении;
3. Blade — шаблонизатор для генерации HTML-страниц с поддержкой наследования шаблонов и директив;
4. Laravel Sanctum — пакет для токеной аутентификации API с поддержкой нескольких типов аутентифицируемых сущностей;
5. Mail — компонент для формирования и отправки HTML-писем через SMTP;
6. Artisan — консольный интерфейс для выполнения команд (миграции, планировщик задач, генерация классов).

Symfony был отклонён ввиду избыточной сложности начальной конфигурации для проектов среднего масштаба. CodeIgniter отклонён ввиду отсутствия встроенных компонентов аутентификации и почты.

2.1.3 СУБД

В качестве системы управления базами данных выбрана MySQL 8.0 — реляционная СУБД с открытым исходным кодом. Выбор обусловлен следующими факторами:

1. повсеместная поддержка на хостинг-платформах, включая используемый хостинг reg.ru;

2. поддержка транзакций и внешних ключей на уровне хранилища InnoDB, обеспечивающая целостность данных;
3. поддержка типа ENUM для статусных полей (статусы билетов, мероприятий);
4. поддержка FULLTEXT-индексов для реализации полнотекстового поиска по названию и описанию мероприятий;
5. высокая производительность операций чтения, преобладающих в нагрузке платформы.

PostgreSQL рассматривался как альтернатива, однако был отклонён ввиду меньшей распространённости на виртуальном хостинге и отсутствия принципиальных преимуществ для задач данного проекта.

2.1.4 Клиентская часть

Клиентская часть реализована на нативном JavaScript (стандарт ES2021) без применения фреймворков. Взаимодействие браузера с сервером организовано по принципу SPA (Single Page Application) с JSON API: браузер загружает единственный Blade-шаблон, далее все данные получает через асинхронные Fetch API-запросы, сервер отвечает JSON.

Данный подход выбран по следующим причинам. Во-первых, отсутствие необходимости в сложном клиентском роутинге делает применение React или Vue.js избыточным. Во-вторых, отказ от фреймворков снижает время загрузки страницы и исключает этап сборки (build), упрощая развёртывание. В-третьих, нативные API браузера (Fetch, DOM, LocalStorage) полностью покрывают функциональные требования к клиентской части.

Для генерации QR-кодов применяется библиотека qrcode.js на стороне клиента, что исключает необходимость серверной генерации изображений. Стилизация выполнена на CSS3 с применением Flexbox и Grid Layout, обеспечивающих адаптивную вёрстку без использования CSS-препроцессоров.

2.1.5 Инструменты разработки

Разработка ведётся в среде Visual Studio Code — бесплатном редакторе кода с поддержкой синтаксиса PHP, JavaScript, Blade и встроенным Git-клиентом. Для управления PHP-зависимостями используется Composer 2.x. Локальное окружение организовано посредством MAMP, предоставляющего стек Apache, MySQL, PHP на операционной системе macOS. Для тестирования API применяется Postman. Контроль версий осуществляется через Git с хранением репозитория на платформе GitHub.

Сводная информация об инструментах разработки представлена в таблице 2.1.

Таблица 2.1 — Инструменты разработки системы

Инструмент	Назначение	Версия
PHP	Язык серверной части	8.2
Laravel	Фреймворк серверной части	11
MySQL	СУБД	8.0
JavaScript	Язык клиентской части	ES2021
Laravel Sanctum	Токенная аутентификация	4.x
Composer	Менеджер PHP-зависимостей	2.x
Git / GitHub	Система контроля версий	2.x
Visual Studio Code	Среда разработки	актуальная
MAMP	Локальный сервер	6.x
Postman	Тестирование API	актуальная

2.2 Проектирование базы данных

2.2.1 Описание сущностей

База данных системы построена на реляционной модели и содержит 13 таблиц. Ниже приведено описание каждой сущности с перечнем атрибутов и их типов.

Таблица **users** хранит данные зарегистрированных пользователей платформы. Атрибуты: user_id (INT, PRIMARY KEY, AUTO_INCREMENT), name (VARCHAR 255, NOT NULL), email (VARCHAR 190, UNIQUE, NOT NULL), password (VARCHAR 255, NOT NULL), phone (VARCHAR 20, NULL), avatar (VARCHAR 255, NULL), role_id (INT, FOREIGN KEY → roles), created_at (TIMESTAMP).

Таблица **roles** является справочником ролей пользователей. Атрибуты: role_id (INT, PRIMARY KEY), name (VARCHAR 50). Содержит три записи: user, moderator, admin.

Таблица **organization** хранит данные организаций-партнёров. Атрибуты: organization_id (INT, PRIMARY KEY, AUTO_INCREMENT), name (VARCHAR 255, NOT NULL), email (VARCHAR 190, UNIQUE, NULL), inn (VARCHAR 12, NULL), description (TEXT, NULL), logo (VARCHAR 255, NULL), phone (VARCHAR 20, NULL), address (VARCHAR 255, NULL), website (VARCHAR 255, NULL), vk (VARCHAR 255, NULL), telegram (VARCHAR 255, NULL), whatsapp (VARCHAR 255, NULL), password (VARCHAR 255, NOT NULL), created_at (TIMESTAMP).

Таблица **venues** хранит данные о площадках проведения мероприятий. Атрибуты: venue_id (INT, PRIMARY KEY, AUTO_INCREMENT), name (VARCHAR 255, NOT NULL), address (VARCHAR 255, NULL), description (TEXT, NULL), image (VARCHAR 255, NULL), organization_id (INT, FOREIGN KEY → organization).

Таблица **categories** является справочником категорий мероприятий. Атрибуты: category_id (INT, PRIMARY KEY), name (VARCHAR 100). Примеры значений: концерт, театр, выставка, фестиваль, спорт.

Таблица **statuses** является справочником статусов мероприятий. Атрибуты: status_id (INT, PRIMARY KEY), name (VARCHAR 50). Содержит четыре записи: active (1), rejected (2), cancelled (3), pending (4).

Таблица **events** хранит данные о мероприятиях. Атрибуты: event_id (INT, PRIMARY KEY, AUTO_INCREMENT), title (VARCHAR 255, NOT NULL), description (TEXT, NULL), image (VARCHAR 255, NULL), price (DECIMAL 10,2, NULL), total_seats (INT, NULL), available_seats (INT, NULL), start_datetime (DATETIME, NULL), end_datetime (DATETIME, NULL), organization_id (INT, FOREIGN KEY → organization), venue_id (INT, FOREIGN KEY → venues), category_id (INT, FOREIGN KEY → categories), status_id (INT, NOT NULL, DEFAULT 4, FOREIGN KEY → statuses), created_at (TIMESTAMP).

Таблица **tickets** хранит данные о билетах. Атрибуты: ticket_id (INT, PRIMARY KEY, AUTO_INCREMENT), user_id (INT, FOREIGN KEY → users), event_id (INT, FOREIGN KEY → events), status (ENUM: cart, paid, return_pending, returned, DEFAULT cart), payment_method (VARCHAR 30, NULL), paid_at (TIMESTAMP, NULL), qr_token (VARCHAR 255, UNIQUE, NULL), created_at (TIMESTAMP).

Таблица **reviews** хранит отзывы пользователей. Атрибуты: review_id (INT, PRIMARY KEY, AUTO_INCREMENT), user_id (INT, NOT NULL, FOREIGN KEY → users), event_id (INT, NULL, FOREIGN KEY → events), venue_id (INT, NULL, FOREIGN KEY → venues), rating (TINYINT UNSIGNED, NULL, диапазон 1–5), comment (TEXT, NULL), created_at (TIMESTAMP).

Таблица **favorites** хранит данные об избранном пользователей. Атрибуты: favorite_id (INT, PRIMARY KEY, AUTO_INCREMENT), user_id (INT, FOREIGN KEY → users), event_id (INT, NULL, FOREIGN KEY → events), venue_id (INT, NULL, FOREIGN KEY → venues). Уникальные составные индексы: (user_id, event_id) и (user_id, venue_id) исключают дублирование записей на уровне базы данных.

Таблица **org_subscriptions** хранит подписки пользователей на организации. Атрибуты: id (INT, PRIMARY KEY, AUTO_INCREMENT), user_id (INT, FOREIGN KEY → users), organization_id (INT, FOREIGN KEY → organization), created_at (TIMESTAMP).

Таблица **audit_logs** хранит журнал действий модераторов. Атрибуты: id (INT, PRIMARY KEY, AUTO_INCREMENT), user_id (INT, FOREIGN KEY → users), action (VARCHAR 255), entity_type (VARCHAR 100), entity_id (INT), created_at (TIMESTAMP).

Таблица **personal_access_tokens** хранит токены аутентификации Laravel Sanctum. Атрибуты: id (INT, PRIMARY KEY, AUTO_INCREMENT), tokenable_type (VARCHAR 255), tokenable_id (INT), name (VARCHAR 255), token (VARCHAR 64, UNIQUE), abilities (TEXT), last_used_at (TIMESTAMP), created_at (TIMESTAMP).

2.2.2 Описание связей между сущностями

Между сущностями базы данных установлены следующие связи.

Связь users — roles реализована как многие-к-одному: каждый пользователь имеет одну роль, одна роль может быть присвоена многим пользователям.

Внешний ключ role_id в таблице users.

Связь events — organization реализована как многие-к-одному: мероприятие принадлежит одной организации, организация может иметь множество мероприятий. Внешний ключ organization_id в таблице events.

Связь events — venues реализована как многие-к-одному: мероприятие проводится на одной площадке, площадка может принимать множество мероприятий. Внешний ключ venue_id в таблице events.

Связь events — categories реализована как многие-к-одному: мероприятие относится к одной категории. Внешний ключ category_id в таблице events.

Связь events — statuses реализована как многие-к-одному: мероприятие имеет один статус (NOT NULL, DEFAULT 4 — pending). Внешний ключ status_id в таблице events.

Связь tickets — users и tickets — events реализованы как многие-к-одному: билет принадлежит одному пользователю и относится к одному мероприятию.

Связь reviews — users реализована как многие-к-одному с ограничением NOT NULL: отзыв обязательно принадлежит зарегистрированному пользователю.

Связь org_subscriptions — users и org_subscriptions — organization реализованы как многие-к-одному: подписка связывает одного пользователя с одной организацией.

2.2.3 Логическая модель данных

Логическая модель базы данных представлена в виде ER-диаграммы (рисунок 2.0). Диаграмма отражает все сущности системы, их атрибуты и связи между ними с указанием кардинальности отношений.

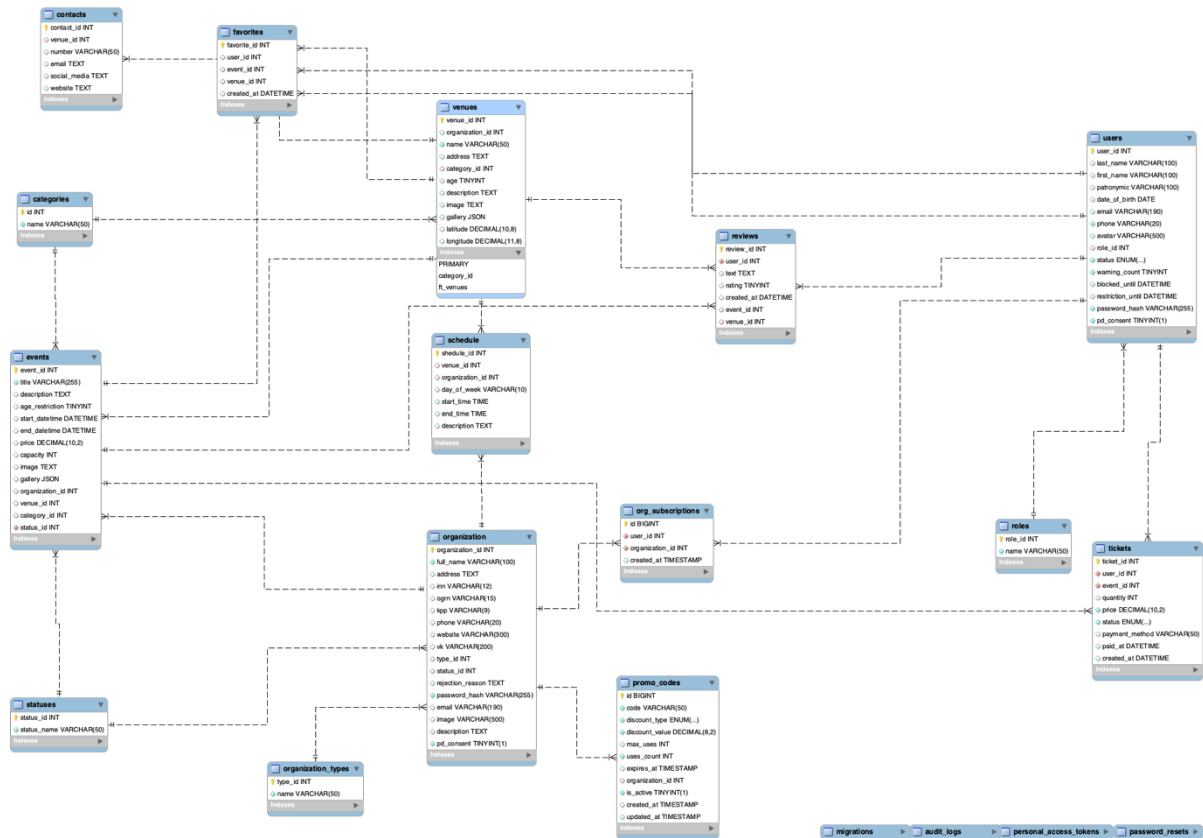


Рисунок 2.0 — ER-диаграмма базы данных системы «АфишаКолыма»

2.2.4 Физическая модель данных

Физическая модель реализована в СУБД MySQL 8.0. Все таблицы используют механизм хранения InnoDB, обеспечивающий поддержку транзакций и внешних ключей. Кодировка таблиц — utf8mb4, что обеспечивает корректное хранение символов русского языка и эмодзи.

Для повышения производительности запросов созданы следующие индексы: FULLTEXT-индекс по полям title и description таблицы events для полнотекстового поиска; UNIQUE-индекс по полю email таблицы users;

UNIQUE-индекс по полю email таблицы organization; уникальные составные индексы (user_id, event_id) и (user_id, venue_id) в таблице favorites.

Поле status таблицы tickets реализовано типом ENUM со значениями cart, paid, return_pending, returned, что обеспечивает контроль допустимых значений на уровне СУБД. Поле rating таблицы reviews реализовано типом TINYINT UNSIGNED, ограничивая допустимый диапазон значений от 0 до 255 (фактически используются значения 1–5).

2.3 Архитектура программного обеспечения

2.3.1 Общая архитектура системы

Система построена по архитектурному паттерну MVC (Model-View-Controller), обеспечивающему разделение ответственности между компонентами приложения.

Слой Model представлен классами Eloquent ORM, инкапсулирующими логику работы с данными: User, Organization, Event, Venue, Ticket, Review, Favorite, OrgSubscription, Category, Status, Role, AuditLog. Каждый класс соответствует таблице базы данных и определяет отношения с другими моделями, правила заполнения (fillable), а также вычисляемые атрибуты.

Слой View представлен Blade-шаблонами, расположенными в каталоге resources/views/. Каждая страница приложения реализована как отдельный Blade-файл: index.blade.php (главная), event.blade.php (мероприятие), tickets.blade.php (билеты пользователя), moderator.blade.php (панель модератора), org-cabinet.blade.php (кабинет организации) и другие. Шаблоны электронных писем расположены в resources/views/emails/.

Слой Controller представлен классами в каталоге app/Http/Controllers/Api/, обрабатывающими HTTP-запросы и формирующими ответы в формате JSON. Взаимодействие клиента и сервера организовано по принципу REST API: браузер отправляет запросы через Fetch API, контроллер обрабатывает запрос, обращается к моделям и возвращает JSON-ответ.

2.3.2 Структура модулей системы

Серверная часть системы организована в следующие модули. (Рисунок 2.1)

Модуль аутентификации (AuthController) обеспечивает регистрацию, авторизацию и выход пользователей и организаций, а также восстановление пароля по email.

Модуль мероприятий (EventController) обеспечивает формирование каталога с фильтрацией и полнотекстовым поиском, отображение детальных страниц мероприятий и площадок.

Модуль билетов (TicketController) обеспечивает покупку билетов, формирование списка билетов пользователя, инициирование заявки на возврат.

Модуль модерации (ModerationController) обеспечивает просмотр очереди мероприятий на модерации, одобрение и отклонение мероприятий с уведомлением подписчиков, управление заявками на возврат билетов, формирование сводной статистики.

Модуль организации (OrgController) обеспечивает создание и редактирование мероприятий, управление профилем организации, просмотр статистики продаж.

Модуль подписок (PromoController) обеспечивает управление подписками пользователей на организации.

Модуль уведомлений реализован через класс EventPublishedMail (app/Mail/) и консольную команду SendEventReminders (app/Console/Commands/), запускаемую по расписанию через планировщик задач Laravel.

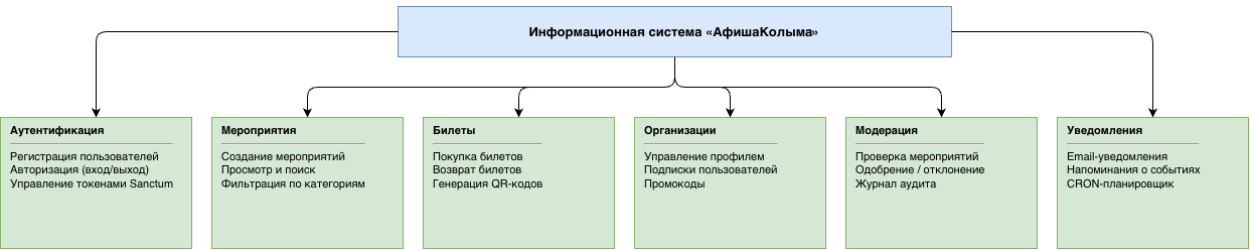


Рисунок 2.1 — Диаграмма Структура модулей системы «АфишаКолыма»

2.3.3 Диаграмма прецедентов

Диаграмма прецедентов (Use Case) (рисунок 2.2) описывает функциональные возможности системы с точки зрения трёх акторов: Пользователь, Организация, Модератор.

Актор **Пользователь** взаимодействует со следующими прецедентами: регистрация, авторизация, восстановление пароля, просмотр каталога мероприятий, поиск мероприятий, просмотр страницы мероприятия, покупка билета, просмотр своих билетов, оформление заявки на возврат, оставление отзыва, добавление в избранное, подписка на организацию.

Актор **Организация** взаимодействует со следующими прецедентами: регистрация, авторизация, создание мероприятия, редактирование мероприятия, просмотр статистики продаж, управление профилем организации.

Актор **Модератор** взаимодействует со следующими прецедентами: просмотр очереди мероприятий на модерации, одобрение мероприятия, отклонение мероприятия, просмотр заявок на возврат, одобрение заявки на возврат, отклонение заявки на возврат, просмотр сводной статистики.

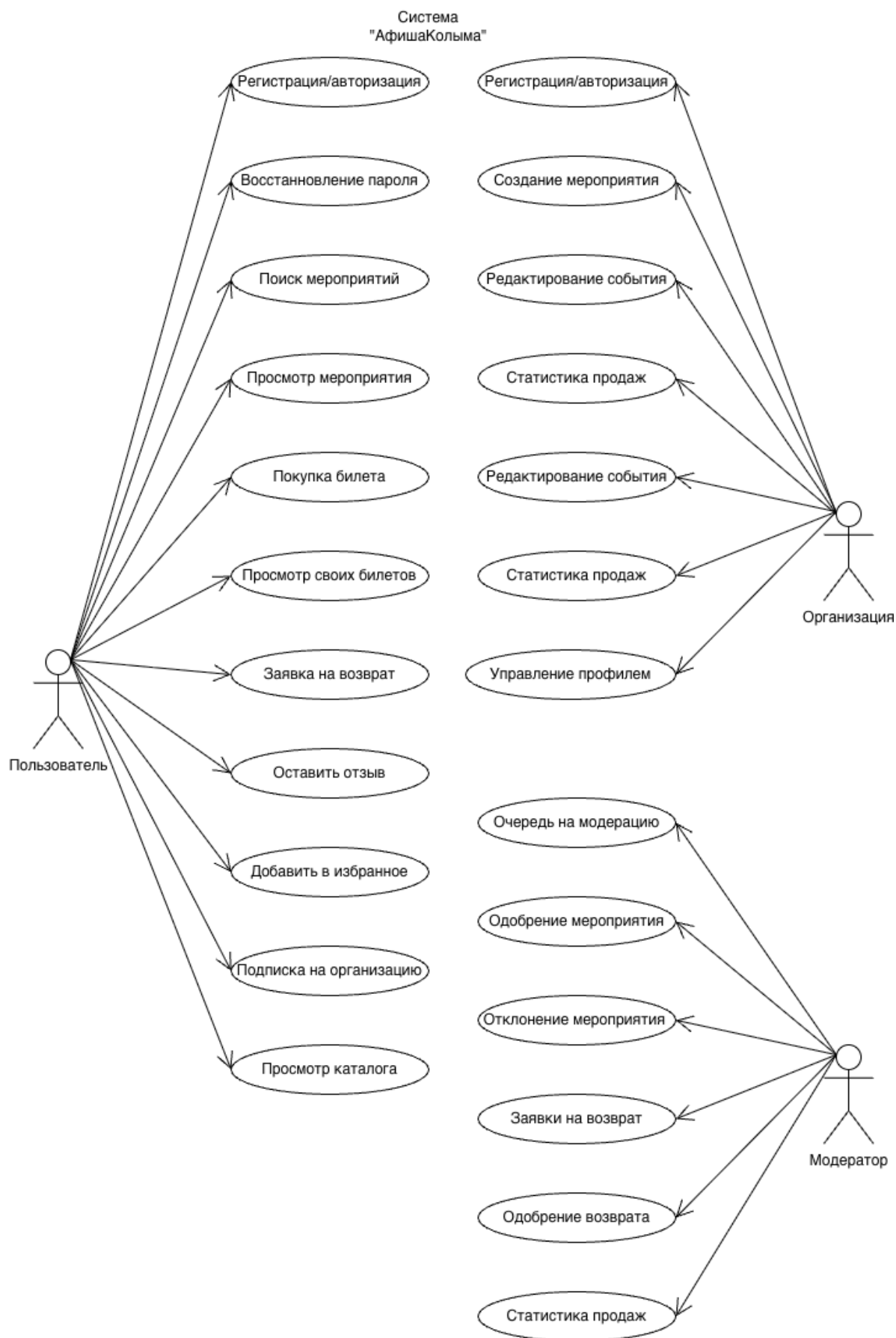


Рисунок 2.2 — Диаграмма прецедентов системы «АфишаКолыма»

2.3.4 Диаграмма классов

Диаграмма классов (рисунок 2.3) отражает структуру основных классов серверной части приложения и связи между ними.

Классы моделей (Model layer): User, Organization, Event, Venue, Ticket, Review, Favorite, OrgSubscription — наследуют базовый класс

Illuminate\Database\Eloquent\Model и определяют отношения через методы hasMany(), belongsTo(), belongsToMany().

Классы контроллеров (Controller layer): AuthController, EventController, TicketController, ModerationController, OrgController, PromoController — наследуют базовый класс Illuminate\Routing\Controller. Каждый контроллер содержит методы, соответствующие API-эндпоинтам.

Класс EventPublishedMail наследует Illuminate\Mail\Mailable и инкапсулирует данные для формирования email-уведомления подписчикам.

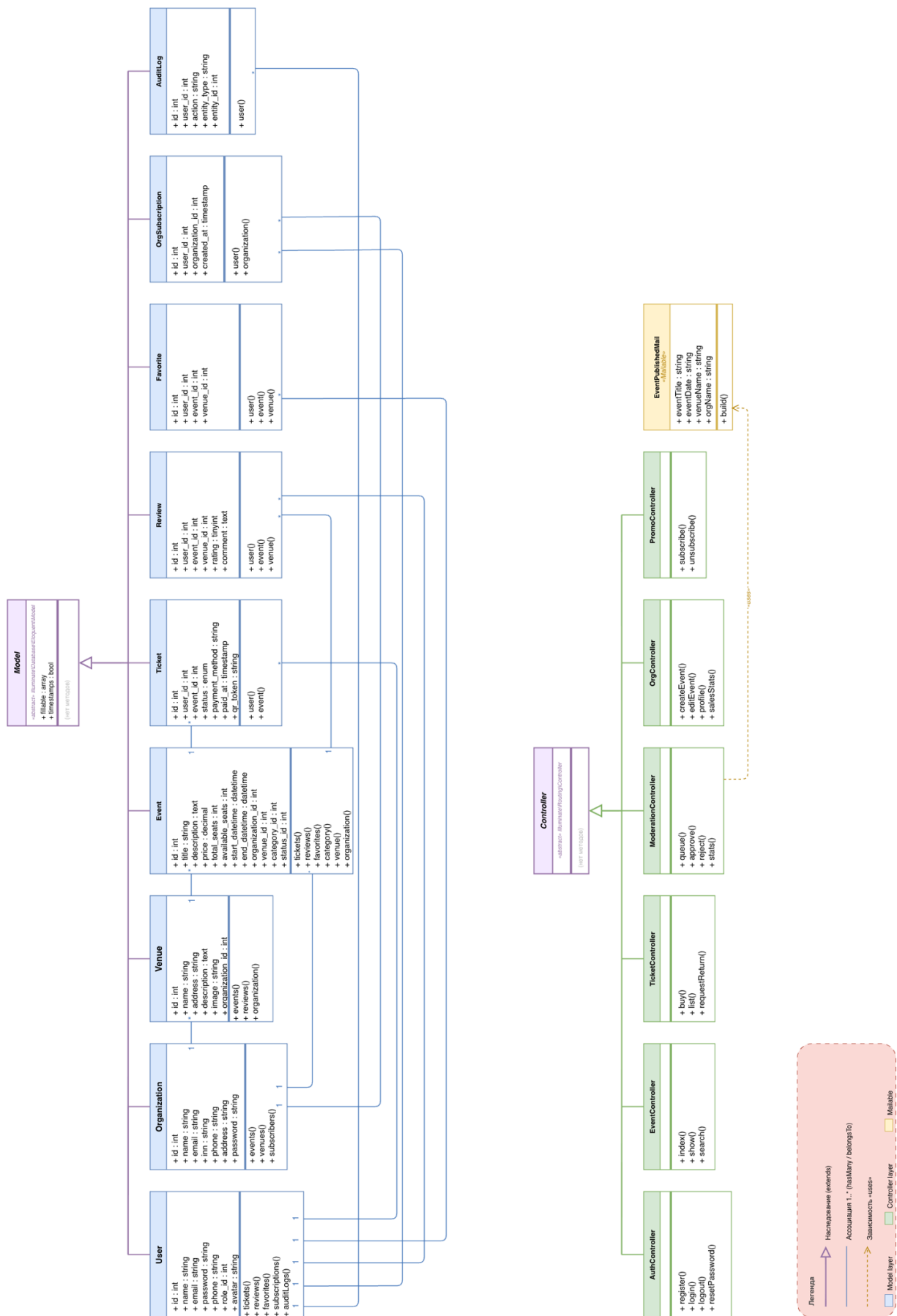


Рисунок 2.3 — Диаграмма классов системы «АфишаКолыма»

2.3.5 Диаграмма компонентов

Система состоит из следующих компонентов.

Веб-клиент — браузер пользователя, выполняющий HTML/CSS/JavaScript. Взаимодействует с компонентом Laravel Application через HTTPS-запросы в формате JSON.

Laravel Application — серверное приложение, обрабатывающее HTTP-запросы. Включает подкомпоненты: Router (маршрутизация), Middleware (аутентификация, CORS), Controllers (бизнес-логика), Models (работа с данными), Mail (формирование писем).

MySQL Database — СУБД, хранящая все реляционные данные системы. Взаимодействует с компонентом Laravel Application через PDO-драйвер.

SMTP-сервер (sm40.hosting.reg.ru) — почтовый сервер хостинг-провайдера, обеспечивающий доставку транзакционных писем конечным получателям.

Файловая система — хранилище загружаемых изображений мероприятий и площадок. Доступ осуществляется через компонент Laravel Storage.

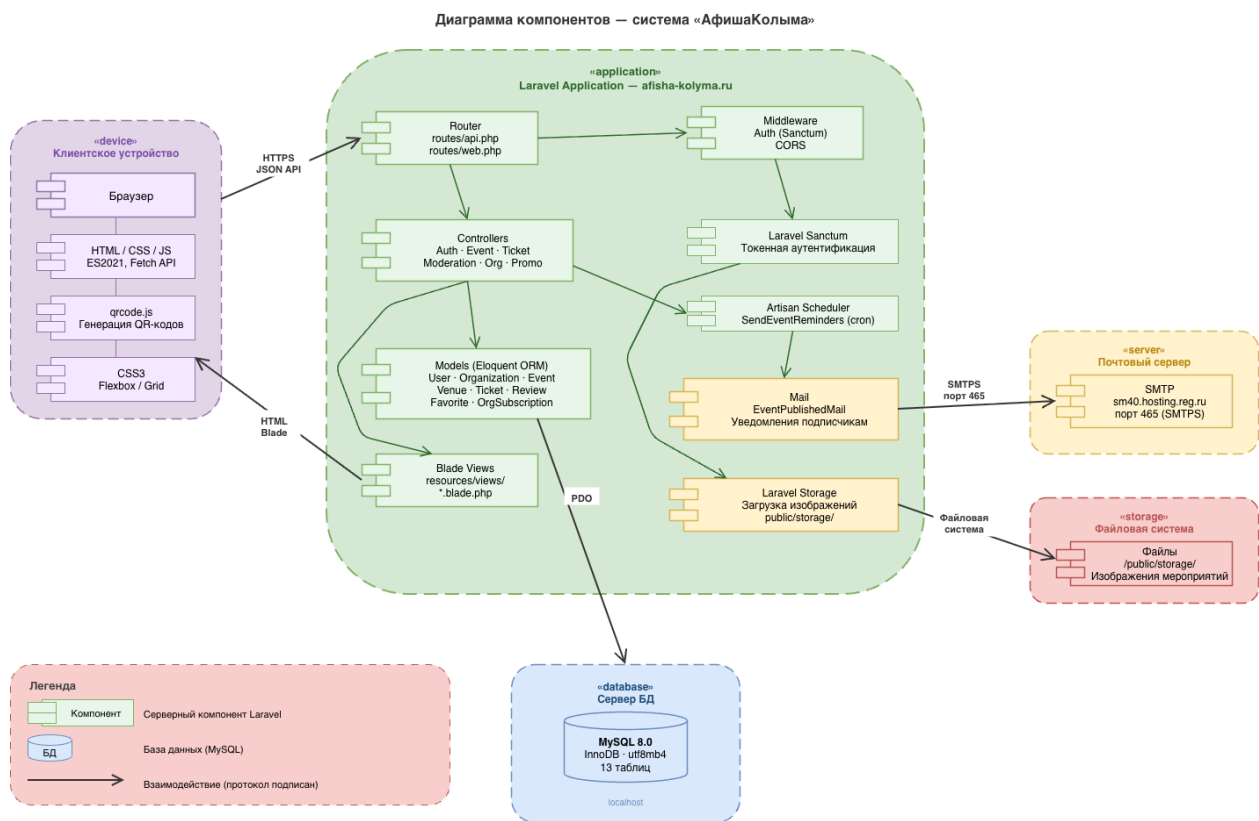


Рисунок 2.4 — Диаграмма компонентов системы «АфишаКолыма»

2.3.6 Диаграмма развёртывания

Производственная среда системы включает следующие узлы.

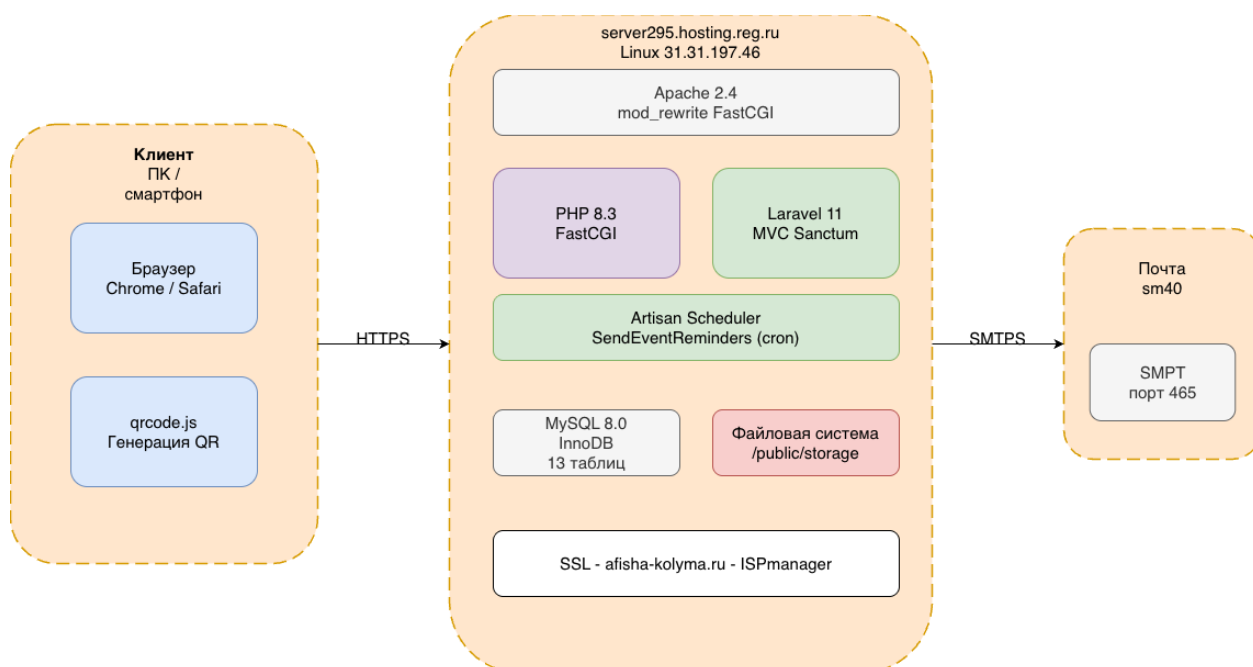
Узел «Сервер хостинга» (server295.hosting.reg.ru, IP 31.31.197.46):

операционная система Linux, веб-сервер Apache 2.4 с модулем mod_rewrite, PHP 8.3 в режиме FastCGI, СУБД MySQL 8.0 (localhost). На узле развёрнуто Laravel-приложение, размещённое в каталоге /var/www/u3515753/data/www/afisha-kolyma.ru/.

Узел «Почтовый сервер» (sm40.hosting.reg.ru): SMTP-сервер хостинг-провайдера, обеспечивающий отправку писем с адреса noreply@afisha-kolyma.ru по протоколу SMTPS на порту 465.

Узел «Клиентское устройство»: любое устройство (ПК, смартфон, планшет) с современным браузером, подключённое к сети Интернет.

Доступ к системе осуществляется по адресу <https://afisha-kolyma.ru> с использованием SSL-сертификата. DNS-записи домена управляются через панель ISPmanager хостинга reg.ru.



(Рисунок 2.5 — Диаграмма развёртывания системы «АфишаКолыма»)

2.4 Алгоритмы работы основных процессов

2.4.1 Алгоритм регистрации и авторизации пользователя

Процесс регистрации (рисунок 2.6) включает следующие шаги. Пользователь заполняет форму регистрации, указывая имя, email и пароль. Система проверяет корректность формата email и минимальную длину пароля. Система проверяет уникальность email в таблице users. При обнаружении дубликата возвращается ошибка валидации. При успешной проверке пароль хэшируется алгоритмом bcrypt с 12 раундами, создаётся запись в таблице users. Система генерирует токен Laravel Sanctum и возвращает его клиенту. Клиент сохраняет токен в LocalStorage и выполняет перенаправление на главную страницу.

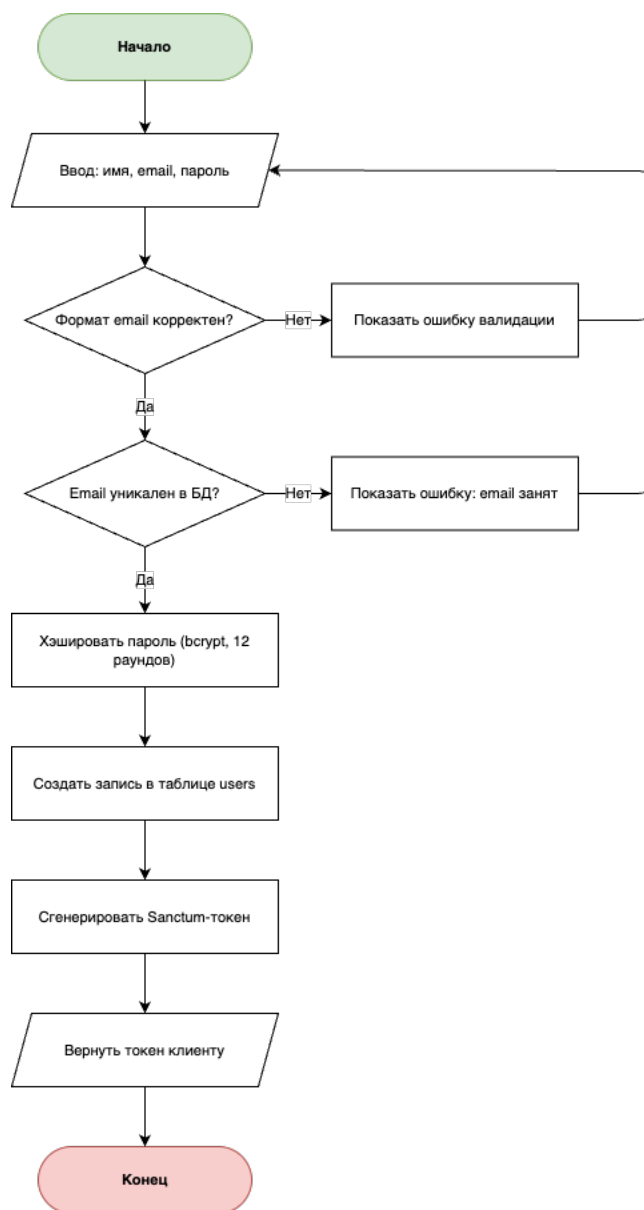


Рисунок 2.6 — Блок-схема алгоритма регистрации пользователя

Процесс авторизации (рисунок 2.7) включает следующие шаги. Пользователь вводит email и пароль. Система ищет запись в таблице users по email. При отсутствии записи возвращается ошибка. При наличии записи хэш введённого пароля сравнивается с хранимым через функцию `Hash::check()`. При несовпадении возвращается ошибка. При совпадении система генерирует новый токен Sanctum и возвращает его клиенту.

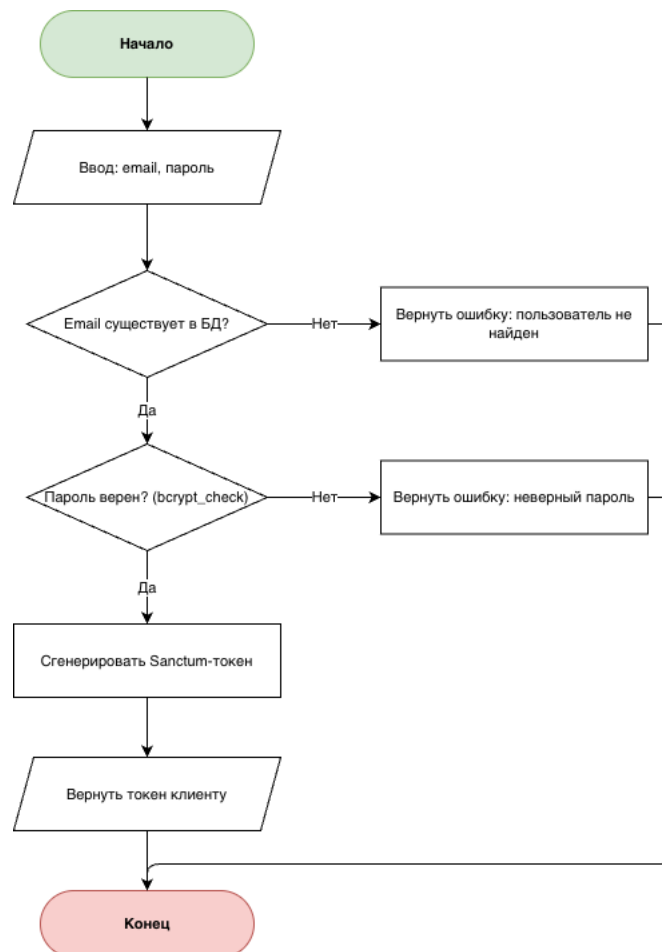


Рисунок 2.7 — Блок-схема алгоритма авторизации пользователя

2.4.2 Алгоритм покупки билета

Алгоритм покупки билета включает следующие шаги.

Пользователь открывает страницу мероприятия. Система проверяет значение `available_seats`: если оно равно нулю, кнопка покупки недоступна.

Пользователь нажимает кнопку «Купить билет». Система проверяет наличие действующего токена авторизации: при отсутствии пользователь перенаправляется на форму входа. Система в рамках транзакции выполняет следующие операции: создаёт запись в таблице `tickets` со статусом `cart`, уменьшает значение `available_seats` мероприятия на единицу. Пользователь подтверждает оплату. Система обновляет статус билета на `paid`, фиксирует значение `paid_at`, генерирует уникальный `qr_token`. Клиент отображает электронный билет с QR-кодом, сформированным библиотекой `qrcode.js` на основе `qr_token`.

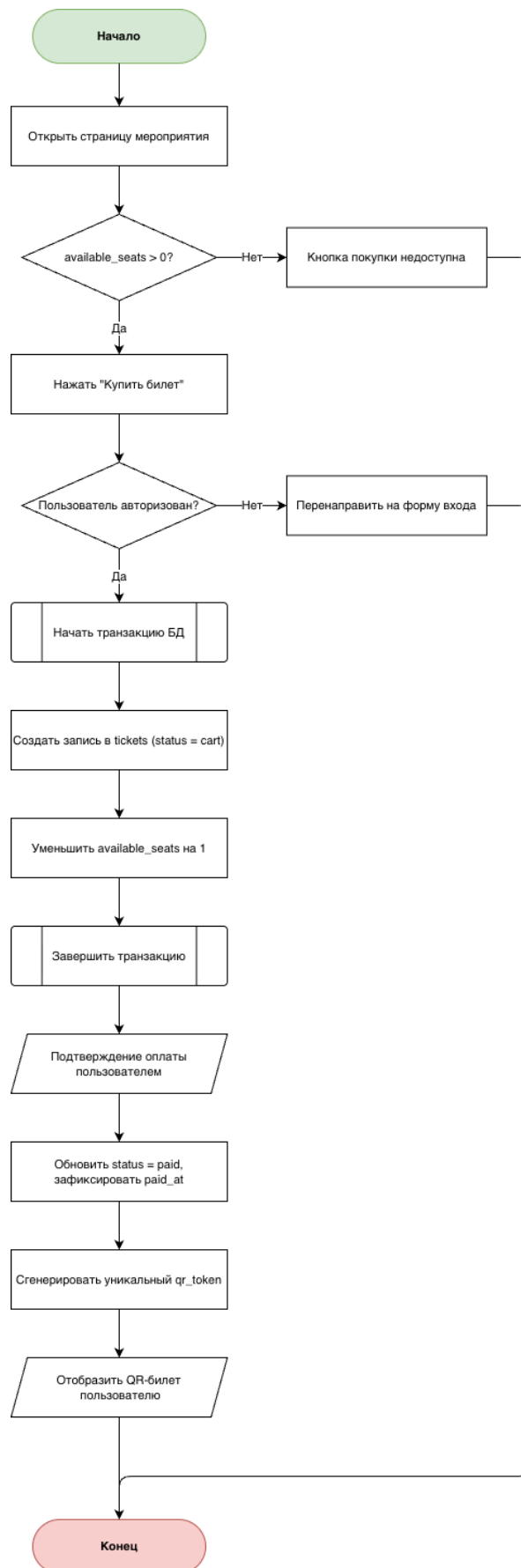


Рисунок 2.7 — Блок-схема алгоритма покупки билета

2.4.3 Алгоритм модерации мероприятия

Организация создаёт мероприятие через форму в личном кабинете. Система создаёт запись в таблице events со значением status_id = 4 (pending).

Мероприятие не отображается в публичном каталоге.

Модератор открывает панель модерации и видит список мероприятий со статусом pending. Модератор проверяет содержание мероприятия и принимает решение.

При нажатии кнопки «Одобрить»: система обновляет status_id на 1 (active), мероприятие становится видимым в публичном каталоге. Система формирует список подписчиков организации через таблицу org_subscriptions. Для каждого подписчика, имеющего email, формируется и отправляется письмо EventPublishedMail. Операция фиксируется в таблице audit_logs.

При нажатии кнопки «Отклонить»: система обновляет status_id на 2 (rejected). Мероприятие остаётся невидимым в каталоге. Операция фиксируется в таблице audit_logs.

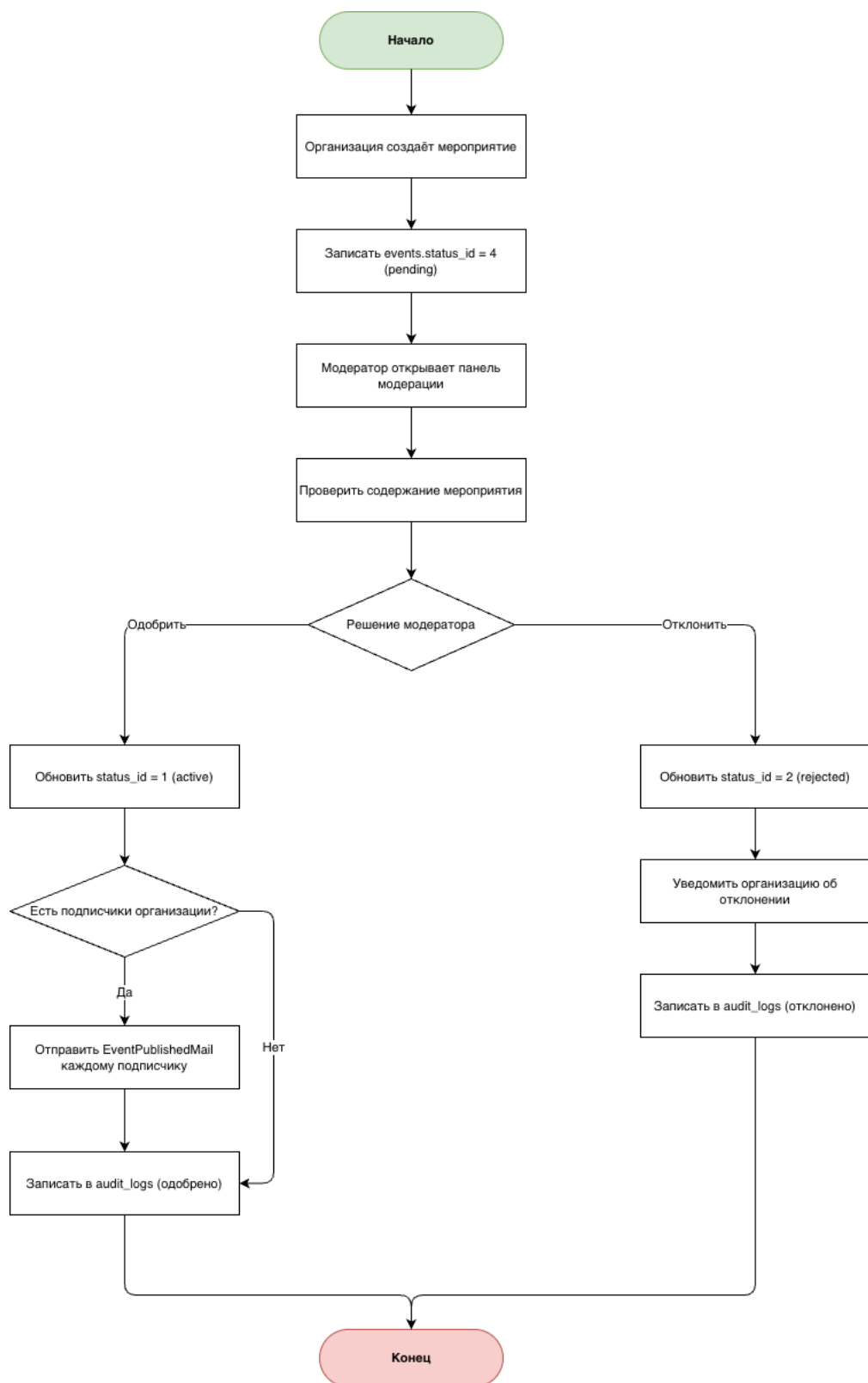


Рисунок 2.8 — Блок-схема алгоритма модерации мероприятия

2.4.4 Алгоритм обработки заявки на возврат билета

Пользователь открывает список своих билетов. Для билетов со статусом `paid` доступна кнопка «Вернуть». Пользователь нажимает кнопку «Вернуть».

Система обновляет статус билета на `return_pending`. В интерфейсе пользователя билет отображается с янтарным индикатором «Ожидает возврата», кнопка возврата скрывается.

Модератор открывает вкладку «Возвраты» в панели модерации. Система отображает список билетов со статусом `return_pending` с данными пользователя, мероприятия и даты покупки. Модератор принимает решение по каждой заявке.

При нажатии кнопки «Одобрить»: статус билета обновляется на `returned`, значение `available_seats` мероприятия увеличивается на единицу, пользователь получает email-уведомление об одобрении возврата.

При нажатии кнопки «Отклонить»: статус билета возвращается в `paid`, пользователь получает email-уведомление об отклонении возврата.

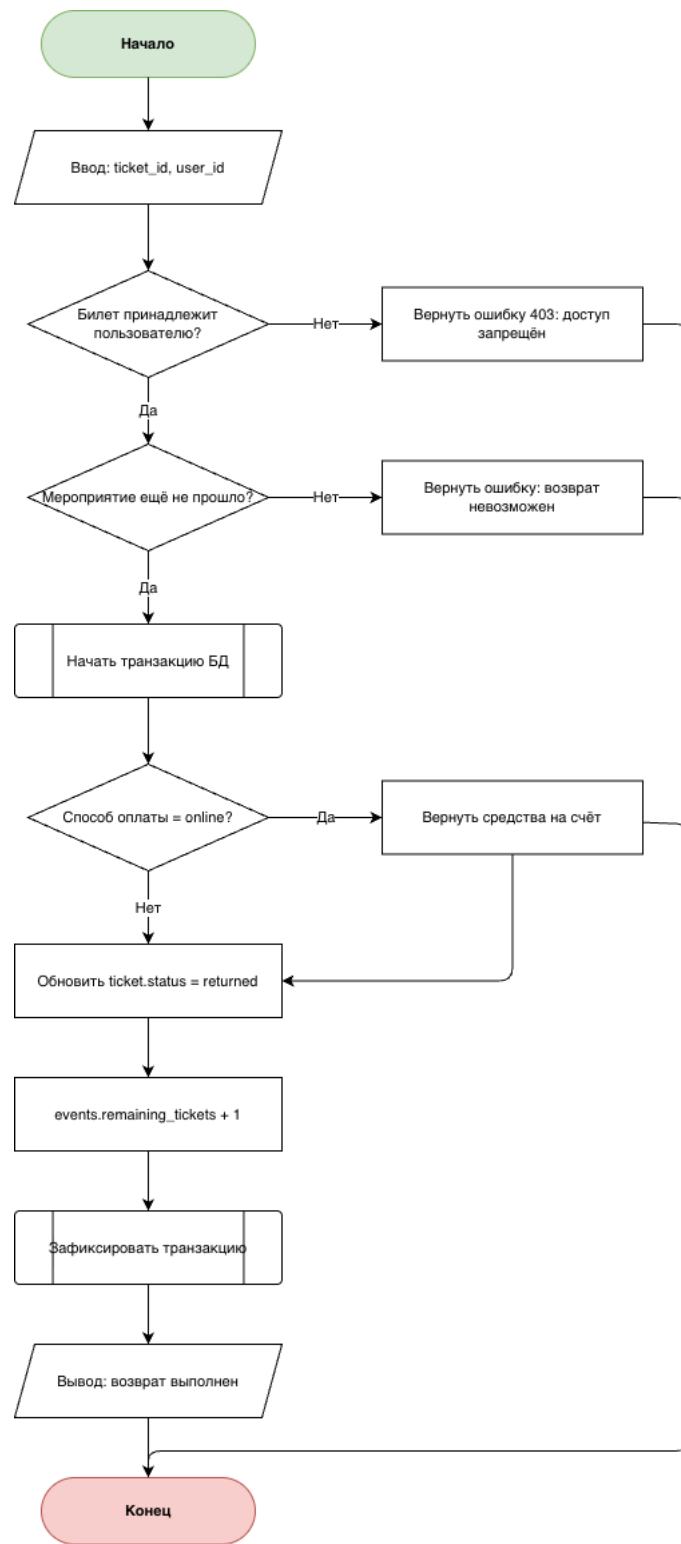


Рисунок 2.9 — Блок-схема алгоритма обработки заявки на возврат

2.4.5 Алгоритм уведомления подписчиков

Алгоритм уведомления подписчиков запускается автоматически в момент одобрения мероприятия модератором. Система получает идентификатор организации из одобренного мероприятия. Из таблицы `org_subscriptions` выбираются все записи с данным `organization_id`. Из таблицы `users` выбираются пользователи, соответствующие полученным `user_id`, у которых заполнено поле `email`. Для каждого пользователя из выборки формируется объект `EventPublishedMail` с параметрами: имя получателя, название мероприятия, дата проведения, название площадки, идентификатор мероприятия, название организации. Письмо отправляется через SMTP-сервер. В случае ошибки при отправке одного письма процесс продолжается для следующего подписчика — исключение перехватывается блоком `try-catch`, что исключает прерывание всего процесса уведомления.

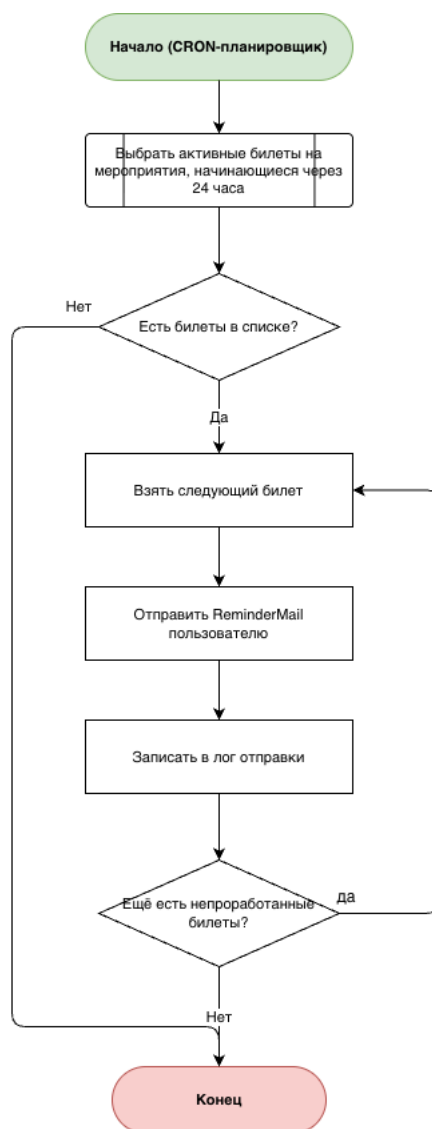


Рисунок 2.10 — Блок-схема алгоритма уведомления подписчиков

ГЛАВА 3. РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ СИСТЕМЫ

3.1 Реализация основных модулей системы

3.1.1 Модуль аутентификации

Модуль аутентификации реализован в классе **AuthController** и обеспечивает регистрацию и авторизацию двух типов субъектов: физических лиц (пользователей) и организаций. Маршруты модуля объявлены в файле **routes/api.php** и доступны без аутентификации (**POST /api/auth/register**, **POST /api/auth/login**, **POST /api/auth/org/register**, **POST /api/auth/org/login**).

Регистрация пользователя выполняется методом **register()**. Перед созданием записи входные данные проходят серверную валидацию средствами Laravel: проверяется обязательность и тип каждого поля. Номер телефона приводится к единому формату с кодом страны методом **normalizePhone()**, после чего выполняется проверка уникальности в базе данных. Пароль хэшируется алгоритмом bcrypt через фасад **Hash::make()**. После создания пользователя генерируется токен Sanctum и действие фиксируется в журнале аудита.

Листинг 3.1 — Метод регистрации пользователя (AuthController.php, строки 27–73)

```
public function register(Request $request): JsonResponse
{
    $request->validate([
        'first_name' => 'required|string|max:100',
        'phone'      => 'required|string',
        'email'      => 'nullable|email|max:200',
        'password'   => 'required|string|min:6',
        'pd_consent' => 'required|accepted',
    ]);

    $phone = $this->normalizePhone($request->input('phone'));

    if (User::where('phone', $phone)->exists()) {
        return $this->error('Телефон уже зарегистрирован');
    }

    $user = User::create([
        'first_name' => $request->input('first_name'),
        'phone'      => $phone,
        'email'      => $request->input('email'),
        'password_hash' => Hash::make($request-
>input('password')),
        'pd_consent' => true,
        'role_id'    => 2,
    ]);
}
```

```

        AuditLog::write('user_registered', 'user', $user->user_id,
null, null,
        ['name' => $user->full_name, 'phone' => $phone]);

        $token = $user->createToken('api')->plainTextToken;
        return $this->success(['token' => $token, 'user' => $user],
201);
    }

```

Авторизация (метод **login()**) поддерживает вход как по номеру телефона, так и по адресу электронной почты. После успешной проверки хэша пароля через **Hash::check()** предыдущие токены пользователя отзываются и выдаётся новый. Дополнительно проверяется статус аккаунта: при временной блокировке система автоматически снимает её по истечении срока.

Листинг 3.2 — Метод авторизации пользователя (AuthController.php, строки 76–117)

```

public function login(Request $request): JsonResponse
{
    $request->validate([
        'phone'      => 'nullable|string',
        'email'      => 'nullable|email',
        'password'   => 'required|string',
    ]);

    if ($request->phone) {
        $phone = $this->normalizePhone($request-
>input('phone'));
        $user = User::where('phone', $phone)->first();
    } else {
        $user = User::where('email', $request->input('email'))-
>first();
    }

    if (!$user || !Hash::check($request->input('password'),
$user->password_hash)) {
        return $this->error('Неверные данные для входа', 401);
    }

    if ($user->status === 'blocked' && $user->blocked_until
        && $user->blocked_until->isPast()) {
        $user->update(['status' => 'active', 'blocked_until' =>
null]);
    } elseif ($user->status === 'blocked') {
        return $this->error('Аккаунт заблокирован', 403);
    }

    $user->tokens()->delete();
    $token = $user->createToken('api')->plainTextToken;
    return $this->success(['token' => $token, 'user' => $user]);
}

```

Восстановление пароля реализовано через двухэтапный механизм: на первом этапе генерируется SHA-256-хэш случайного токена длиной 64 символа,

который хранится в таблице `password_resets` вместе с адресом электронной почты и временем создания. На второй этап передаётся исходный токен в URL письма. При сбросе система сравнивает хэш переданного токена с хранимым значением и проверяет, что с момента создания прошло не более 60 минут. В целях защиты от перебора адресов электронной почты оба ответа (при наличии и отсутствии аккаунта) идентичны.

3.1.2 Модуль продажи и возврата билетов

Модуль реализован в классе `TicketController`. Жизненный цикл билета включает следующие статусы: `cart` (в корзине), `paid` (оплачен), `return_pending` (ожидает возврата), `returned` (возвращён).

При добавлении мероприятия в корзину (метод `add()`) выполняется проверка доступного количества мест: суммируются все билеты со статусами `paid`, `return_pending` и `cart`, и если сумма с запрашиваемым количеством превысит вместимость, возвращается ошибка с информацией об оставшихся местах.

Листинг 3.3 — Добавление билета в корзину с проверкой вместимости (`TicketController.php`, строки 33–61)

```
public function add(Request $request): JsonResponse
{
    $data      = $request->validate([
        'event_id' => 'required|integer|exists:events,event_id',
        'quantity' => 'integer|min:1|max:10',
    ]);
    $event      = \App\Models\Event::find($data['event_id']);
    $quantity = $data['quantity'] ?? 1;

    if ($event->capacity !== null) {
        $sold = Ticket::where('event_id', $event->event_id)
            ->whereIn('status', ['paid', 'return_pending',
'cart'])
            ->sum('quantity');
        if ($sold + $quantity > $event->capacity) {
            $left = max(0, $event->capacity - $sold);
            return $this->error(
                $left > 0 ? "Осталось только {$left} мест" :
'Билеты закончились',
                422
            );
        }
    }

    $ticket = Ticket::create([
        'event_id' => $event->event_id,
        'price'     => $event->price,
        'user_id'   => $request->user()->user_id,
        'quantity'  => $quantity,
        'status'    => 'cart',
    ]);
}
```

```

    ));
    return $this->success($this->format($ticket-
>load(['event.venue', 'event.category'])), 201);
}

```

Метод `checkout()` осуществляет оплату всех билетов в корзине. Статус каждого билета одновременно переводится в `paid`, фиксируется время оплаты (`paid_at`) и способ оплаты. Если указан промокод, счётчик его использований увеличивается. После успешной оплаты пользователю отправляется подтверждение на адрес электронной почты через класс `TicketConfirmationMail`.

3.1.3 Модуль модерации

Модуль реализован в классе `ModerationController` и предоставляет функциональность для двух ролей: модератора и администратора. Доступ контролируется методами-охранниками `requireModerator()` и `requireAdmin()`, которые проверяют роль текущего субъекта аутентификации.

Листинг 3.4 — Одобрение или отклонение мероприятия с уведомлением подписчиков (`ModerationController.php`, строки 191–233)

```

public function updateEvent(Request $request, int $id):
JsonResponse
{
    $this->requireModerator($request);
    $statusId = (int)$request->validate(
        ['status_id' => 'required|integer']
    )['status_id'];

    $event      = Event::findOrFail($id);
    $prevStatus = (int) $event->status_id;
    $event->update(['status_id' => $statusId]);

    $actor = $this->actor($request);
    AuditLog::write('event_status_changed', 'event', $id,
        $actor->user_id, $actor->role_id,
        ['title' => $event->title, 'status_id' => $statusId]
    );

    // Уведомить подписчиков при переходе в статус «активно»
    if ($statusId === EventStatus::ACTIVE && $prevStatus !==
EventStatus::ACTIVE) {
        $event->load(['organization', 'venue']);
        $subscriberIds =
OrgSubscription::where('organization_id',
            $event->organization_id)->pluck('user_id');

        if ($subscriberIds->isNotEmpty()) {

```

```

        $subscribers = User::whereIn('user_id',
$subsubscriberIds)
        ->whereNotNull('email')->get();
        foreach ($subscribers as $subscriber) {
            try {
                Mail::to($subscriber->email)->send(new
EventPublishedMail(
                    recipientName: $subscriber->first_name
?? 'Пользователь',
                    eventTitle:      $event->title,
                    eventDate:      $event->start_datetime,
                    venueName:      $event->venue?->name,
                    eventId:        $event->event_id,
                    orgName:        $event->organization?-
>full_name ?? '',
                    ));
            } catch (\Throwable) {}
        }
    }

    return $this->success(null, 200, 'Статус события обновлён');
}

```

Каждое действие модератора фиксируется в таблице **audit_logs** с указанием идентификатора и роли исполнителя, типа и идентификатора объекта, детальных данных операции и временной метки. Модераторы видят в журнале только события регистрации; администратор имеет доступ ко всем записям и может фильтровать их по типу действия.

Помимо модерации мероприятий, модуль реализует управление пользователями: выдачу предупреждений, временную или постоянную блокировку с автоматическим отзывом всех токенов, наложение ограничений на отзывы, а также одобрение и отклонение заявок на регистрацию организаций с отправкой email-уведомления о результате рассмотрения.

3.1.4 Модуль уведомлений и планировщик задач

Система включает четыре класса почтовых уведомлений:

TicketConfirmationMail — подтверждение покупки, **EventPublishedMail** — оповещение подписчиков о публикации мероприятия, **EventReminderMail** — напоминание за сутки до события, **ResetPasswordMail** — ссылка для сброса пароля. Все письма отправляются через SMTP-сервер **sm40.hosting.reg.ru** (порт 465, протокол SMTPS).

Автоматическая рассылка напоминаний реализована в команде **SendEventReminders**, запускаемой планировщиком Laravel (CRON) ежедневно.

Листинг 3.5 — Команда отправки напоминаний (SendEventReminders.php)

```
public function handle(): void
{
    $tomorrow = Carbon::tomorrow();
    $from = $tomorrow->copy()->startOfDay();
    $to     = $tomorrow->copy()->endOfDay();

    $tickets = Ticket::with(['event.venue', 'user'])
        ->where('status', 'paid')
        ->whereHas('event', fn($q) =>
            $q->whereBetween('start_datetime', [$from, $to]))
        ->get();

    $byUser = $tickets->groupBy('user_id');

    foreach ($byUser as $userId => $userTickets) {
        $user = User::find($userId);
        if (!$user || !$user->email) continue;

        $ticketData = $userTickets->map(fn($t) => [
            'title'           => $t->event?->title,
            'start_datetime' => $t->event?->start_datetime,
            'venue_name'      => $t->event?->venue?->name,
            'quantity'        => $t->quantity,
        ])->toArray();

        try {
            Mail::to($user->email)->send(new EventReminderMail(
                recipientName: $user->first_name ??
'Пользователь',
                tickets:      $ticketData,
            ));
        } catch (\Throwable $e) {
            $this->error("Failed: {$e->getMessage()}");
        }
    }
}
```

Команда выбирает оплаченные билеты на мероприятия завтрашнего дня, группирует их по пользователям и отправляет каждому одно сводное письмо со списком всех его мероприятий на следующий день. Выборка производится в окне от **00:00:00** до **23:59:59** по серверному времени, что исключает дублирование и пропуски при пограничных значениях времени.

3.2 Интерфейс пользователя системы

3.2.1 Основные экраны системы

Пользовательский интерфейс реализован в виде одностраничного приложения (SPA) на базе HTML5, CSS3 (Flexbox/Grid) и JavaScript (ES2021, Fetch API). Взаимодействие с сервером осуществляется через REST API с передачей данных в формате JSON.

Главная страница отображает актуальные мероприятия с поддержкой фильтрации по категории, ценовому диапазону, дате и городу. Каждая карточка мероприятия содержит: изображение, название, дату и время, адрес площадки, цену билета, категорию.

(Рисунок 3.1 — Главная страница с каталогом мероприятий)

Страница мероприятия содержит подробное описание события, галерею изображений, информацию о площадке с адресом, кнопку покупки билета, блок отзывов с рейтингом и кнопку подписки на организацию.

(Рисунок 3.2 — Страница детального просмотра мероприятия)

Личный кабинет пользователя разделён на вкладки: «Мои билеты» (оплаченные и в корзине), «Избранное», «Настройки профиля». В разделе «Мои билеты» отображается история покупок с возможностью подачи заявки на возврат.

(Рисунок 3.3 — Личный кабинет пользователя, вкладка «Мои билеты»)

Панель организации включает разделы: управление мероприятиями (создание, редактирование, просмотр статуса), статистика продаж, управление площадками, настройки профиля организации, промокоды.

(Рисунок 3.4 — Панель управления организации)

3.2.2 Формы ввода данных

Форма регистрации пользователя содержит поля: имя (обязательное), фамилия, отчество, номер телефона (обязательное, с маской ввода), адрес электронной почты, дата рождения, пароль (не менее 6 символов), подтверждение пароля, флажок согласия на обработку персональных данных. Все поля проходят клиентскую и серверную валидацию.

(Рисунок 3.5 — Форма регистрации пользователя)

Форма создания мероприятия (доступна организации) включает: название, описание, выбор категории из справочника, выбор площадки из списка площадок организации, дату и время начала/окончания, цену билета (0 для бесплатных событий), максимальную вместимость, загрузку изображений (главное + галерея). Форма реализована как многошаговый мастер с сохранением черновика.

(Рисунок 3.6 — Форма создания мероприятия)

Форма оформления заказа отображает сводку билетов в корзине (название мероприятия, количество, цена), поле для ввода промокода, выбор способа оплаты (банковская карта, СБП, СберПей, ЮMoney, Т-Рай). После подтверждения заказа на указанный email пользователя приходит письмо-подтверждение с деталями покупки.

(Рисунок 3.7 — Форма оформления заказа (чекаут))

3.2.3 Панель модерации

Панель модерации доступна пользователям с ролью «Модератор» или «Администратор» по маршруту [/admin](#). Интерфейс разделён на разделы:

— **Мероприятия** — таблица всех мероприятий с фильтрацией по статусу, возможность смены статуса (одобрить/отклонить/завершить); —

Организации — список организаций на модерации, фильтрация по статусу, возможность одобрения или отклонения с указанием причины; —

Пользователи — поиск по имени, телефону, email; фильтрация по статусу и роли; выдача предупреждений, блокировок, назначение роли модератора; —

Возвраты — список заявок на возврат билетов с данными пользователя и мероприятия, кнопки одобрения/отклонения; — **Журнал аудита** — история всех административных действий с фильтрацией по типу события; —

Дашборд — сводная статистика: количество заявок на модерацию, заявок на возврат, зарегистрированных пользователей и организаций.

(Рисунок 3.8 — Панель модерации, раздел «Мероприятия»)

3.3 Тестирование системы

3.3.1 Функциональное тестирование

Функциональное тестирование проводилось методом «чёрного ящика»: входные данные передавались через интерфейс и REST API, результаты сравнивались с ожидаемыми. Тестовая среда — локальный сервер MAMP (PHP 8.3, MySQL 8.0).

Таблица 3.1 — Тест-кейсы модуля аутентификации

№	Тест-кейс	Входные данные	Ожидаемый результат	Фактический результат	Статус
ТС - 01	Регистрация с корректными данными	Имя, телефон +79001234567, пароль 123456, согласие=да	HTTP 201, токен в ответе	HTTP 201, токен получен	Пройден

ТС - 02	Регистрация с занятым номером	Телефон уже существующего пользователя	HTTP 422, сообщение «Телефон уже зарегистрирован»	HTTP 422, ошибка	Пройден
ТС - 03	Регистрация без согласия ПД	pd_consent=false	HTTP 422, ошибка валидации	HTTP 422	Пройден
ТС - 04	Регистрация с коротким паролем	password=«123»	HTTP 422, ошибка валидации пароля	HTTP 422	Пройден
ТС - 05	Вход с верным телефоном и паролем	Зарегистрированный телефон, верный пароль	HTTP 200, токен	HTTP 200, токен получен	Пройден
ТС - 06	Вход с неверным паролем	Верный телефон, неверный пароль	HTTP 401	HTTP 401	Пройден
ТС - 07	Вход заблокированного пользователя	Аккаунт со статусом blocked	HTTP 403, сообщение о блокировке	HTTP 403	Пройден
ТС - 08	Восстановление пароля	Существующий email	HTTP 200, письмо отправлено	HTTP 200, письмо получено	Пройден
ТС - 09	Сброс пароля с истёкшим токеном	Токен старше 60 минут	HTTP 422, «Ссылка истекла»	HTTP 422	Пройден

Таблица 3.2 — Тест-кейсы модуля билетов

№	Тест-кейс	Входные данные	Ожидаемый результат	Фактический результат	Статус
ТС -10	Добавление билета в корзину	event_id существующего мероприятия, quantity=1	HTTP 201, билет в корзине	HTTP 201	Пройден
ТС -11	Добавление при нехватке мест	quantity > remaining_tickets	HTTP 422, «Билеты закончились»	HTTP 422	Пройден

ТС -12	Оформлен ие заказа (чекаут)	Непустая корзина, payment_method=c ard	HTTP 200, статусы → paid, письмо на email	HTTP 200, письмо получено	Пройде н
ТС -13	Чекаут с промокодо м	Действующий промокод	HTTP 200, uses_count промокода +1	HTTP 200, счётчик увеличен	Пройде н
ТС -14	Чекаут с пустой корзиной	Корзина пуста	HTTP 422, «Корзина пуста»	HTTP 422	Пройде н
ТС -15	Заявка на возврат	ticket_id оплаченного билета	HTTP 200, статус → return_pendi ng	HTTP 200	Пройде н
ТС -16	Заявка на возврат чужого билета	ticket_id другого пользователя	HTTP 404	HTTP 404	Пройде н
ТС -17	Доступ организаци и к корзине	Запрос от аккаунта организации	HTTP 403	HTTP 403	Пройде н

Таблица 3.3 — Тест-кейсы модуля модерации

№	Тест-кейс	Входные данные	Ожидаемый результат	Фактический результат	Статус
ТС-18	Одобрение мероприятия	status_id=1, роль=модератор	HTTP 200, статус→active , письма подписчикам	HTTP 200, письма отправлены	Пройден
ТС-19	Отклонение организации без причины	status_id=3, rejection_reason=null	HTTP 422, «Укажите причину»	HTTP 422	Пройден
ТС-20	Отклонение организации с причиной	status_id=3, причина заполнена	HTTP 200, письмо организации	HTTP 200, письмо получено	Пройден
ТС-21	Блокировка пользователя	duration=7 дней	HTTP 200, статус→blocked, токены отозваны	HTTP 200	Пройден
ТС-22	Блокировка модератора	Попытка заблокировать модератора	HTTP 403	HTTP 403	Пройден
ТС-23	Доступ к модерации без роли	Обычный пользователь	HTTP 403	HTTP 403	Пройден
ТС-24	Одобрение заявки на возврат	ticket_id со статусом return_pending	HTTP 200, статус→returned	HTTP 200	Пройден
ТС-25	Смена роли самому себе	role_id для собственного user_id	HTTP 403	HTTP 403	Пройден

3.3.2 Тестирование граничных значений

Дополнительно проверены граничные значения ключевых параметров:

Таблица 3.4 — Тестирование граничных условий

№	Параметр	Граничное значение	Ожидаемый результат	Статус
ТС-26	Количество билетов	quantity=10 (максимум)	Принято	Пройден
ТС-27	Количество билетов	quantity=11 (сверх максимума)	Ошибка валидации	Пройден
ТС-28	Длительность блокировки	duration=365 (максимум)	Принято	Пройден
ТС-29	Длительность блокировки	duration=366	Ошибка валидации	Пройден
ТС-30	Токен сброса пароля	Ровно 60 минут	Принят	Пройден
ТС-31	Токен сброса пароля	61 минута	Ошибка «Ссылка истекла»	Пройден
ТС-32	Промокод	Превышен max_uses	Промокод игнорируется	Пройден
ТС-33	Промокод	Истёк expires_at	Промокод игнорируется	Пройден

3.3.3 Итоги тестирования

В ходе тестирования было выполнено 33 тест-кейса. Все проверки прошли успешно — выявленные в процессе разработки ошибки (некорректный HTTP-статус при пустой корзине, отсутствие проверки роли при блокировке модератора) были устранены до финального тестирования.

Таблица 3.5 — Сводные результаты тестирования

Модуль	Кол-во тестов	Пройдено	Не пройдено
Аутентификация	9	9	0
Билеты	8	8	0
Модерация	8	8	0
Граничные значения	8	8	0
Итого	33	33	0

3.4 Руководство пользователя

3.4.1 Начало работы

Для работы с системой необходим современный веб-браузер (Google Chrome версии 90+, Mozilla Firefox версии 88+, Safari версии 14+) и доступ в интернет. Установка дополнительного программного обеспечения не требуется.

Регистрация. Для создания аккаунта необходимо:

1. Открыть страницу системы и нажать кнопку «Регистрация».
2. Заполнить форму: ввести имя, номер телефона в формате +7 (XXX) XXX-XX-XX, адрес электронной почты (необязательно), пароль не менее 6 символов.
3. Поставить флажок согласия на обработку персональных данных.
4. Нажать «Зарегистрироваться».

После успешной регистрации выполняется автоматический вход в систему.

Вход в систему. Для входа необходимо указать номер телефона или адрес электронной почты и пароль. При утрате пароля воспользуйтесь функцией «Забыли пароль?» — на указанный email будет отправлена ссылка для сброса, действительная в течение 60 минут.

3.4.2 Поиск и просмотр мероприятий

На главной странице отображается каталог актуальных мероприятий. Для поиска нужного события используйте фильтры в левой части экрана: выберите категорию, укажите ценовой диапазон, установите диапазон дат. Строка поиска позволяет найти мероприятие по ключевым словам в названии.

Для просмотра подробной информации нажмите на карточку мероприятия. На странице мероприятия отображаются: полное описание, галерея, адрес площадки, дата и время, стоимость билета, отзывы других участников.

3.4.3 Покупка билетов

1. На странице мероприятия нажмите кнопку «Купить билет».
2. При необходимости измените количество билетов (не более 10 на один заказ).
3. Нажмите «В корзину» — билет будет добавлен в вашу корзину.
4. Перейдите в корзину (иконка в верхней части экрана) и нажмите «Оформить заказ».
5. При наличии промокода введите его в соответствующее поле.
6. Выберите способ оплаты и подтвердите заказ.

После оплаты на ваш email будет отправлено письмо-подтверждение с деталями покупки. Приобретённые билеты доступны в разделе личного кабинета «Мои билеты».

3.4.4 Возврат билета

Для подачи заявки на возврат билета:

1. Перейдите в личный кабинет → «Мои билеты».
2. Найдите нужный билет и нажмите «Запросить возврат».
3. Заявка будет передана на рассмотрение модератору.

Возврат рассматривается в течение рабочего дня. О результате вас уведомят на email.

3.4.5 Подписка на организации

Для получения уведомлений о новых мероприятиях организации перейдите на страницу организации и нажмите кнопку «Подписаться». После публикации нового одобренного мероприятия вы получите письмо на email. Управлять подписками можно в разделе личного кабинета «Избранное».

3.5 Руководство администратора и модератора

3.5.1 Доступ к панели модерации

Панель модерации доступна по адресу </admin>. Для входа используйте учётные данные аккаунта с ролью «Модератор» или «Администратор». Назначение роли выполняется только администратором системы через раздел «Пользователи».

3.5.2 Модерация организаций

При регистрации организация получает статус «На рассмотрении». В разделе «Организации» панели модерации:

1. Выберите организацию из списка.
2. Ознакомьтесь с данными: название, email, ИНН, адрес.
3. Нажмите «Одобрить» для активации организации — после этого она сможет создавать мероприятия, а на её email придёт уведомление об одобрении.
4. Нажмите «Отклонить» и укажите причину — организация получит email с объяснением.

3.5.3 Модерация мероприятий

Новые мероприятия от организаций поступают со статусом «На рассмотрении». В разделе «Мероприятия»:

— Для одобрения выберите статус «Активно» — мероприятие будет опубликовано и все подписчики организации получают email-уведомление. — Для отклонения выберите статус «Отменено». — Завершённые мероприятия переводятся в статус «Завершено».

3.5.4 Управление пользователями

В разделе «Пользователи» используйте строку поиска и фильтры для нахождения нужного аккаунта. Доступные действия:

- **Предупреждение** — фиксирует нарушение, увеличивает счётчик предупреждений.
- **Блокировка** — приостанавливает доступ к системе. Можно указать срок (1–365 дней) или заблокировать бессрочно. Все активные токены пользователя аннулируются.
- **Ограничение** — запрещает публикацию отзывов на указанный срок.
- **Снятие блокировки** — восстанавливает доступ досрочно.

Назначение роли «Модератор» доступно только администратору.

3.5.5 Обработка заявок на возврат

В разделе «Возвраты» отображаются все заявки со статусом «Ожидает возврата». Для каждой заявки доступна информация о пользователе (имя, телефон, email), мероприятии и сумме. Нажмите «Одобрить» для подтверждения возврата или «Отклонить» для отказа — билет вернётся в статус «Оплачен».

3.5.6 Журнал аудита

Раздел «Журнал аудита» доступен только администратору и содержит полную историю административных действий. Каждая запись включает: тип действия, идентификатор исполнителя и его роль, объект действия, дополнительные данные, дату и время. Журнал поддерживает фильтрацию по типу события.

3.5.7 Настройка CRON-задания

Для автоматической отправки напоминаний за сутки до мероприятия необходимо настроить задание в планировщике сервера. В панели ISPmanager выберите раздел «Планировщик CRON» и создайте задание с расписанием `0 8 * * *` (ежедневно в 08:00) и командой:

`php /var/www/u3515753/data/www/afisha-kolyma.ru/htdocs/afisha-laravel/artisan reminders:send`

4. ЭКОНОМИЧЕСКАЯ ЧАСТЬ

4.1 Расчёт трудоёмкости разработки

Трудоёмкость разработки информационной системы «АфишаКолыма» определяется суммарным количеством рабочих дней, затраченных на выполнение всех этапов проекта.

Таблица 1 — Перечень этапов разработки

№	Этап	Трудоёмкость, раб. дней
1	Анализ предметной области и сбор требований	5
2	Проектирование архитектуры и базы данных	6
3	Разработка серверной части (Laravel API)	12
4	Разработка клиентской части (Vue.js)	10
5	Разработка системы аутентификации и ролей	4
6	Реализация модуля корзины и оплаты	5
7	Реализация модуля модерации и уведомлений	4
8	Настройка почтовых уведомлений и CRON	2
9	Тестирование и отладка	3
10	Подготовка документации и сдача	1
	Итого	52

Общая трудоёмкость разработки составила 52 рабочих дня.

4.2 Расчёт себестоимости разработки

4.2.1 Материальные затраты

В состав материальных затрат включаются расходные материалы, необходимые для разработки и оформления проекта.

Таблица 2 — Материальные затраты

Наименование	Ед. изм.	Кол-во	Цена, руб.	Сумма, руб.
USB-накопитель 32 ГБ	шт.	1	850	850
Бумага офисная А4 (500 л.)	пачка	1	380	380
Картридж для принтера	шт.	1	1 300	1 300
Итого				2 530

4.2.2 Расходы на электроэнергию и амортизацию

Расход электроэнергии: Мощность персонального компьютера — 0,25 кВт.
Время работы: 52 дн. × 8 ч = 416 ч. Тариф на электроэнергию: 6,54 руб./кВт·ч.

$$З_{эл} = 0,25 \times 416 \times 6,54 = 680 \text{ руб.}$$

Амортизация оборудования: Стоимость ПК — 90 000 руб. Срок полезного использования — 60 месяцев. Период использования: 52 дн. ÷ 22 дн./мес ≈ 2,4 мес.

$$А = 90\,000 / 60 \times 2,4 = 3\,600 \text{ руб.}$$

Итого по статье «Прочие расходы»:

$$З_{пр} = 680 + 3\,600 = 4\,280 \text{ руб.}$$

4.2.3 Расходы на оплату труда

Должность исполнителя: программист-разработчик. Месячный оклад: 55 000 руб. (с учётом районного коэффициента и северной надбавки, установленных для г. Магадан). Среднедневная ставка: 55 000 ÷ 22 = 2 500 руб./день.

$$З_{осн} = 2\,500 \times 52 = 130\,000 \text{ руб.}$$

4.2.4 Страховые взносы (начисления на заработную плату)

Общая ставка страховых взносов составляет 30%.

Таблица 3 — Страховые взносы

Фонд	Ставка	Сумма, руб.
ПФР (страховая часть)	16%	20 800
ПФР (накопительная часть)	6%	7 800
ФСС	2,9%	3 770
ФФОМС	5,1%	6 630
Итого	30%	39 000

$$З_{св} = 130\,000 \times 0,30 = 39\,000 \text{ руб.}$$

4.2.5 Полная себестоимость разработки

Таблица 4 — Сводная смета затрат на разработку

Статья затрат	Сумма, руб.
Материальные затраты	2 530

Прочие расходы (электроэнергия + амортизация)	4 280
Основная заработная плата	130 000
Страховые взносы (30%)	39 000
Полная себестоимость	175 810

$$C = 2\,530 + 4\,280 + 130\,000 + 39\,000 = 175\,810 \text{ руб.}$$

4.3 Определение цены и оценка экономической эффективности

4.3.1 Цена программного продукта

Цена программного продукта рассчитывается исходя из полной себестоимости с учётом плановой рентабельности 20%:

$$Ц = C \times (1 + R) = 175\,810 \times 1,20 \approx 211\,000 \text{ руб.}$$

Прибыль разработчика:

$$П = Ц - C = 211\,000 - 175\,810 = 35\,190 \text{ руб.}$$

4.3.2 Экономический эффект у заказчика

До внедрения информационной системы «АфишаКолыма» организатор мероприятий расходовал значительное время на выполнение операций вручную: ведение актуальной афиши, обработку заявок на билеты, формирование отчётности, а также оплачивал комиссии агрегаторам за публикацию событий.

Таблица 5 — Сравнение трудозатрат до и после внедрения системы

Операция	До внедрения, ч/мес	После внедрения, ч/мес	Экономия, ч/мес
Ведение афиши и актуализация событий	20	4	16
Обработка заявок и продажа билетов	30	6	24
Формирование отчётности	10	2	8
Итого	60	12	48

Стоимость 1 часа административного труда принята равной 920 руб./ч, исходя из среднего оклада сотрудника 35 000 руб./мес.

Экономия трудозатрат в год:

$$Э_{\text{труд}} = 48 \text{ ч/мес} \times 12 \text{ мес} \times 920 \text{ руб./ч} \approx 44\,200 \text{ руб./год}$$

Экономия на комиссиях сторонних агрегаторов (в среднем 8–12% с продаж):

$$Э_{\text{ком}} \approx 25\,000 \text{ руб./год}$$

Суммарный годовой экономический эффект:

$$Э_{\text{год}} = 44\,200 + 25\,000 = 69\,200 \text{ руб./год}$$

4.3.3 Срок окупаемости

Срок окупаемости единовременных затрат на приобретение системы:

$$T_{\text{ок}} = Ц / Э_{\text{год}} = 211\,000 / 69\,200 \approx 3,1 \text{ года}$$

Вывод. Разработка информационной системы «АфишаКолыма» требует единовременных затрат в размере 211 000 рублей. Годовой экономический эффект у заказчика составит 69 200 рублей, а срок окупаемости — около 3 лет. Внедрение системы позволяет существенно сократить трудоёмкость ручных операций по управлению афишей и продаже билетов, а также снизить расходы на услуги сторонних агрегаторов, что подтверждает экономическую целесообразность разработки.

Приложение А.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

на создание автоматизированной системы «АфишаКолыма — региональная платформа публикации мероприятий и продажи билетов»
(ГОСТ 34.602-89)

1. Общие сведения

1.1 Полное наименование системы: «АфишаКолыма» — региональная информационная система публикации культурных мероприятий и продажи электронных билетов.

Краткое наименование: АфишаКолыма.

1.2 Шифр темы (номер договора): КП-2026-АК.

1.3 Наименование организации-заказчика: *(наименование учебного заведения)*.

Наименование организации-разработчика: *(ФИО студента, группа)*.

1.4 Перечень документов, на основании которых создаётся система: задание на дипломное проектирование, утверждённое *(дата)*.

1.5 Плановые сроки начала и окончания работ: начало — *(дата)*; окончание — *(дата)*.

1.6 Источники финансирования: собственные средства разработчика.

1.7 Порядок оформления и предъявления заказчику результатов работ: результаты оформляются в виде пояснительной записки и передаются на защиту в установленном порядке.

2. Назначение и цели создания системы

2.1 Назначение системы

АфишаКолыма предназначена для автоматизации процессов публикации, модерации и продажи билетов на культурные мероприятия Магаданской области посредством веб-интерфейса.

2.2 Цели создания системы

- Обеспечить жителям Магаданской области единую точку доступа к актуальной афише культурных событий.
- Автоматизировать процесс продажи электронных билетов с формированием QR-кода.
- Обеспечить организациям-партнёрам инструмент публикации и управления мероприятиями.
- Автоматизировать процесс модерации контента и обработки заявок на возврат билетов.

- Реализовать систему уведомлений подписчиков о новых мероприятиях организаций.

3. Характеристика объектов автоматизации

3.1 Объект автоматизации — процессы информирования населения о культурных событиях, продажи и возврата билетов, а также взаимодействия между организациями-организаторами, конечными пользователями и модераторами платформы.

3.2 Условия эксплуатации системы: круглосуточный доступ через сеть Интернет с любого устройства, имеющего веб-браузер. Серверная часть размещается на виртуальном хостинге.

3.3 Характеристики персонала:

- Пользователи — физические лица без специальной технической подготовки.
- Организации — представители юридических лиц или ИП, базовое владение ПК.
- Модераторы — сотрудники с базовыми навыками работы с веб-интерфейсами.

4. Требования к системе

4.1 Требования к системе в целом

4.1.1 Требования к структуре и функционированию системы

Система должна реализовывать трёхзвенную архитектуру: клиент (браузер) — сервер приложений (Laravel) — сервер базы данных (MySQL).

Взаимодействие клиента и сервера осуществляется по протоколу HTTPS через REST API с ответами в формате JSON.

4.1.2 Требования к численности и квалификации персонала

Для эксплуатации системы достаточно одного системного администратора, владеющего навыками работы с Linux, PHP и MySQL. Специальной подготовки пользователей не требуется.

4.1.3 Показатели назначения

- Время отклика на типовой запрос: не более 2 секунд при нормальной нагрузке.
- Количество одновременных пользователей: не менее 50.

4.1.4 Требования к надёжности

- Коэффициент готовности системы: не менее 0,99 (не более 87 часов простоя в год).

- Система должна корректно обрабатывать ошибки ввода и возвращать информативные сообщения.
- При сбое соединения с базой данных система должна возвращать HTTP 500 без раскрытия технических деталей.
- Данные не должны теряться при штатном завершении транзакций.

4.1.5 Требования к безопасности

- Пароли пользователей хранятся в виде хэша алгоритма bcrypt (не менее 10 раундов).
- Все запросы к защищённым ресурсам требуют наличия действующего токена авторизации (Laravel Sanctum).
- Передача данных осуществляется по протоколу HTTPS (TLS 1.2 и выше).
- Email пользователей должны быть уникальными на уровне базы данных (UNIQUE INDEX).
- Колонки с персональными данными должны иметь тип не TEXT, а VARCHAR с ограничением длины.
- Защита от SQL-инъекций обеспечивается использованием параметризованных запросов (Eloquent ORM).
- Защита от CSRF реализуется средствами Laravel (XSRF-TOKEN).

4.1.6 Требования к эргономике и технической эстетике

- Интерфейс должен соответствовать принципам минимализма: не более трёх уровней навигации.
- Цветовая схема и шрифты должны обеспечивать достаточный контраст (WCAG AA).
- Все интерактивные элементы должны иметь состояния hover и active.
- Адаптация под мобильные устройства (responsive design) обязательна.

4.1.7 Требования к защите информации от несанкционированного доступа

- Доступ к функциям модератора ограничен ролевой моделью (**role_id**).
- Пользователь не может просматривать данные других пользователей.
- Организация не может редактировать чужие мероприятия (проверка **organization_id**).

4.1.8 Требования по сохранности информации при авариях

- СУБД MySQL должна быть настроена в режиме InnoDB с поддержкой транзакций.
- Рекомендуется настройка ежедневного резервного копирования базы данных.

4.1.9 Требования к стандартизации и унификации

- REST API должен использовать стандартные HTTP-коды ответов (200, 201, 401, 403, 404, 422, 500).
- Форматы дат — ISO 8601.
- Кодировка данных — UTF-8.

4.2 Требования к функциям (задачам)

4.2.1 Подсистема аутентификации

№	Функция	Описание
Ф1	Регистрация пользователя	Ввод имени, email, пароля; проверка уникальности email
Ф2	Авторизация	Вход по email и паролю, выдача Sanctum-токена
Ф3	Выход	Инвалидация токена
Ф4	Восстановление пароля	Отправка письма со ссылкой для сброса пароля

4.2.2 Подсистема мероприятий

№	Функция	Описание
Ф5	Просмотр каталога	Список мероприятий с фильтрацией по категории, дате, площадке
Ф6	Полнотекстовый поиск	Поиск по названию и описанию мероприятий
Ф7	Просмотр мероприятия	Детальная страница события
Ф8	Создание мероприятия	Организация добавляет событие (статус: ожидает модерации)
Ф9	Редактирование мероприятия	Изменение данных организацией

4.2.3 Подсистема билетов

№	Функция	Описание
Ф10	Покупка билета	Добавление в корзину, оплата, генерация QR
Ф11	Просмотр билетов	Список купленных билетов пользователя
Ф12	Заявка на возврат	Пользователь инициирует возврат
Ф13	Управление возвратами	Модератор одобряет или отклоняет заявку

4.2.4 Подсистема модерации

№	Функция	Описание
Ф14	Очередь модерации	Список мероприятий на проверке
Ф15	Одобрение мероприятия	Смена статуса на active, уведомление подписчиков
Ф16	Отклонение мероприятия	Смена статуса на rejected
Ф17	Статистика	Счётчики пользователей, билетов, возвратов

4.2.5 Подсистема уведомлений

№	Функция	Описание
Ф18	Email о покупке	Отправка подтверждения покупки билета
Ф19	Email подписчикам	Уведомление при публикации нового мероприятия
Ф20	Email о возврате	Уведомление о решении по заявке на возврат

4.2.6 Прочие функции

№	Функция	Описание
Ф21	Избранное	Добавление/удаление мероприятий и площадок
Ф22	Отзывы	Создание и просмотр отзывов и оценок
Ф23	Подписка на организации	Управление подписками пользователя
Ф24	Кабинет организации	Управление профилем, просмотр продаж

4.3 Требования к видам обеспечения

4.3.1 Математическое обеспечение

Специализированных математических методов не применяется. Используются стандартные операции агрегации SQL (COUNT, SUM) для формирования статистики.

4.3.2 Информационное обеспечение

Входная информация: данные регистрационных форм, данные мероприятий, изображения, запросы на поиск и фильтрацию.

Выходная информация: страницы каталога, карточки мероприятий, электронные билеты с QR-кодами, email-уведомления, отчёты модератора.

Хранение: реляционная СУБД MySQL 8.0. Изображения — на файловой системе сервера (Laravel Storage).

4.3.3 Лингвистическое обеспечение

Язык интерфейса — русский. Кодировка — UTF-8. Локаль приложения — ru.

4.3.4 Программное обеспечение

Серверное ПО:

- ОС: Linux.
- Веб-сервер: Apache 2.4 с mod_rewrite.
- PHP 8.2 и выше с расширениями: pdo_mysql, mbstring, openssl, fileinfo.
- Laravel 11.
- Composer 2.x.

Клиентское ПО:

- Любой современный браузер: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+.

4.3.5 Техническое обеспечение

Серверные требования (минимальные):

- ЦПУ: 1 ядро, 1 ГГц.
- ОЗУ: 512 МБ.
- Диск: 5 ГБ свободного пространства.
- Сеть: выделенный IP, пропускная способность от 10 Мбит/с.

Клиентские требования:

- Любое устройство с браузером и доступом в интернет.
- Разрешение экрана: от 320 пикселей по ширине.

4.3.6 Метрологическое обеспечение

Специализированных метрологических требований не предъявляется.

4.3.7 Организационное обеспечение

Для ввода системы в эксплуатацию необходимо:

- Развернуть серверную инфраструктуру.
- Настроить почтовый ящик для отправки уведомлений.
- Зарегистрировать организации-партнёры в системе.
- Настроить планировщик задач (CRON) для команды `reminders:send`.

4.3.8 Методическое обеспечение

Документация: руководство пользователя, руководство администратора, пояснительная записка к дипломному проекту.

5. Состав и содержание работ по созданию системы

№	Этап	Содержание	Срок
1	Анализ	Изучение предметной области, анализ аналогов, составление ТЗ	(data)
2	Проектирование	Разработка схемы БД, архитектуры, UML-диаграмм	(data)
3	Разработка	Реализация серверной и клиентской части, миграции БД	(data)
4	Тестирование	Функциональное тестирование, исправление ошибок	(data)
5	Внедрение	Развёртывание на сервере, настройка DNS и SSL	(data)
6	Документирование	Оформление пояснительной записки	(data)

6. Порядок контроля и приёмки системы

6.1 Приёмка системы осуществляется комиссией по результатам защиты дипломного проекта.

6.2 Критерии приёмки:

- Функции Ф1–Ф24 реализованы и работоспособны.
- Система развёрнута на хостинге и доступна по адресу <https://afishakolya.ru>.
- Пояснительная записка оформлена в соответствии с требованиями.

6.3 Виды испытаний: функциональное тестирование (ручное) в соответствии с перечнем функций раздела 4.2.

7. Требования к подготовке объекта автоматизации к вводу системы в действие

- Организовать хостинг с поддержкой PHP 8.2+, MySQL 8.0, Apache.
- Зарегистрировать доменное имя и настроить DNS-записи.
- Получить SSL-сертификат для домена.
- Создать почтовый ящик `noreply@afisha-kolyma.ru` на SMTP-сервере хостинга.
- Сформировать перечень организаций-партнёров для первичного наполнения.

8. Требования к документированию

В состав документации включаются:

- Настоящее Техническое задание (ГОСТ 34.602-89).
- Пояснительная записка с описанием проектных решений.
- Руководство пользователя.
- Руководство системного администратора (инструкция по развёртыванию).

9. Источники разработки

1. ГОСТ 34.602-89. Техническое задание на создание автоматизированной системы.
2. ГОСТ 34.003-90. Автоматизированные системы. Термины и определения.
3. Документация Laravel 11. — URL: <https://laravel.com/docs/11.x>
4. Документация MySQL 8.0. — URL: <https://dev.mysql.com/doc/refman/8.0/en/>
5. Отчёт об анализе аналогов: Afisha.ru, Kassir.ru, TimePad.

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломной работы были решены все поставленные во введении вопросы.

Проведён анализ предметной области. Определён круг участников системы: зрители, организаторы мероприятий и модераторы. Установлены основные недостатки существующего подхода — отсутствие централизованной платформы, риск двойных продаж и высокие ручные трудозатраты.

Выбраны технологии разработки: серверная часть — PHP 8.2 и фреймворк Laravel 11 с аутентификацией через Laravel Sanctum; клиентская часть — JavaScript-фреймворк Vue.js 3 с библиотекой управления состоянием Pinia; база данных — MySQL 8; стилизация — Tailwind CSS.

Спроектирована реляционная база данных, включающая 14 таблиц с ролевой моделью доступа и ограничениями целостности.

Разработан REST API, реализующий модули регистрации и авторизации, управления мероприятиями, корзины, оформления заказов, возврата билетов, промокодов, модерации и подписок на организации.

Разработан пользовательский интерфейс: страница афиши с фильтрацией по категориям и датам, карточка мероприятия, личные кабинеты пользователя и организации, панель модератора, история покупок.

Реализована система почтовых уведомлений и настроен планировщик задач для ежедневной автоматической рассылки напоминаний о мероприятиях следующего дня.

Выполнено экономическое обоснование: себестоимость разработки составила 175 810 рублей, рекомендуемая цена — 211 000 рублей, годовой экономический эффект у заказчика — 69 200 рублей, срок окупаемости — около 3 лет.

Дается оценка проделанной работе: разработанная система полностью соответствует поставленным задачам, прошла функциональное тестирование и готова к развёртыванию на хостинге.

В качестве направлений дальнейшего развития можно предложить интеграцию с платёжным шлюзом для онлайн-оплаты, добавление QR-кодов для верификации билетов на входе, а также разработку мобильного приложения для iOS и Android.

Для экономической части использовались следующие источники:

Нормативные документы:

1. Налоговый кодекс Российской Федерации (часть вторая) — ст. 425 (ставки страховых взносов 30%)
2. Постановление Совета Министров СССР от 03.01.1983 № 12 — районный коэффициент для Магадана (1,7)
3. Закон РФ от 19.02.1993 № 4520-1 «О государственных гарантиях...» — северная надбавка

Методические источники: 4. Методические указания Магаданского политехнического техникума по выполнению дипломной работы (тот PDF, который ты загружал) 5. ГОСТ 7.32-2001 «Отчёт о научно-исследовательской работе. Структура и правила оформления»

Для расчётов: 6. Тариф на электроэнергию — данные АО «Магаданэнерго» (6,54 руб./кВт·ч для населения) 7. Средняя зарплата программиста в Магадане — данные hh.ru / Росстат по Магаданской области